

长程 Agent 工程笔记

Build Agents That Run for Hours (Without Losing the Plot)

Ash Prabaker 与 Andrew Wilson 的长程 Agent Harness 工作坊笔记

AI Engineer / Anthropic Workshop

生成日期: 2026 年 5 月 25 日



视频标题: Build Agents That Run for Hours (Without Losing the Plot) - Ash Prabaker & Andrew Wilson, Anthropic

频道: AI Engineer

发布日期: 2026-05-18

时长: 1:15:40

链接: <https://www.youtube.com/watch?v=mR-WAvEPRwE>

字幕说明: 英文人工字幕不可用, 本笔记使用 YouTube 自动英文字幕并结合画面核对; 原始 SRT 时间戳保留为图像与内容定位依据。

目录

阅读导引：术语与符号速查	3
1 为什么长程 Agent 会丢线	4
1.1 从 20 分钟到数小时：问题的尺度变化	4
1.2 三类失控来源：上下文、规划、判断	5
1.3 模型能力与 Harness 共同演化	6
1.4 本章小结	7
2 Claude Code 演化史：长程运行的原语从哪里来？	8
2.1 从 Artifact、Computer Use 到 MCP	8
2.2 Claude Code：产品也是模型改进的实验场	8
2.3 Ralph Loop：可预测失败的循环	9
2.4 Checkpoints、Sub-agents 与 Agent SDK	10
2.5 Agent Teams 与 Server-side Compaction	11
2.6 本章小结	12
3 当前 Harness 范式：用角色分离压住自评偏差	13
3.1 从自评到对抗式评价	13
3.2 Evaluator 必须真实使用产品	14
3.3 把品味写成 Rubric	15
3.4 重启比补丁更便宜的时刻	16
3.5 本章小结	17
4 Planner 与 Contract：把模糊愿望压成可测试断言	17
4.1 Planner 只画外轮廓	17
4.2 Generator 与 Evaluator 先谈 Done 的定义	19
4.3 文件系统 Hand-off：低技术但可靠的胶水	19
4.4 游戏示例：从一句 AI Features 到可玩的系统	20
4.5 本章小结	23
5 可执行的具体性：27 条 Contract 与 Trace Debugging	23
5.1 CI 没抓到的错误：Evaluator 可以抓到	23
5.2 27 条 Criteria：具体性带来可行动反馈	23
5.3 读 Trace 像读 Stack Trace	26
5.4 用另一个 Agent 辅助分析 Transcript	27
5.5 本章小结	27
6 Frontier 移动后，Harness 也要删减	27
6.1 Harness 是对模型短板的外部补偿	27
6.2 从 Opus 4.5 到 Opus 4.6：删掉多余循环	28
6.3 DAW 示例：能力边界也要写清楚	30
6.4 普通开发者可以复刻的原语	31
6.5 本章小结	32

7 Q&A 上半场：复用、信任、重跑与模型选择	32
7.1 哪些 Rubric 能复用，哪些必须项目化	33
7.2 新上下文循环与单会话压缩如何取舍	33
7.3 看着浏览器跑：信任缺口与自动化验证	34
7.4 重跑、丢弃与继续迭代	34
7.5 Planner 何时回环，何时保持高层	35
7.6 按角色选模型，而非按品牌选模型	35
7.7 本章小结	36
8 Q&A 下半场：Agent Teams、上下文隔离、Brownfield 与人类介入	37
8.1 Agent Teams 与专用 Harness 的边界	37
8.2 Evaluator 看输出，不共享错误思路	37
8.3 Rubric 如何更科学	38
8.4 Greenfield 到 Brownfield 的迁移成本	39
8.5 团队级 Harness、可观测性与版本控制	39
8.6 Human-in-the-loop 的最小插入点	40
8.7 本章小结	40
9 总结：让 Agent 长程可信的三层结构	41
9.1 三层结构：能力、Harness、证据	41
9.2 从一句 Prompt 到可持续工程流程	42
9.3 实践清单：从最小可用 Harness 开始	43
9.4 边界与开放问题	43
9.5 本章小结	44

阅读导引：术语与符号速查

本笔记将英文材料重写为中文教学文本，保留产品名、模型名和少量工程术语。原始字幕为 YouTube 自动英文字幕，时间戳用于定位画面和内容，最终表述以视频语义和画面证据为准。

英文术语	中文写法	解释
long-running agent	长程运行 Agent	能在数小时甚至更久的时间窗内持续推进任务的智能体，核心风险是目标、状态和判断随时间漂移。
Agent Harness	智能体脚手架 / Harness	包在模型外部的执行结构,包括工具循环、权限、上下文注入、子 Agent、skills、hooks、状态文件和评估器。
context window	上下文窗口	模型一次推理能直接看到的信息范围；窗口外的信息需要文件、摘要、检索或新会话重新带回。
context rot	上下文腐化	会话变长后，模型对目标、约束和局部事实的稳定性下降。
context anxiety	上下文焦虑	接近上下文上限时，模型急于收尾，容易把仍需验证的状态写成已经完成。
coherence	连贯性	任务目标、约束和执行状态在长时间内保持一致的能力。
Ralph Loop	Ralph 循环	重复调用 Claude Code CLI，并用计划、退出条件、safe word 和 stop hook 管理循环。
Evaluator	评估器 / Critic	负责真实使用产品、运行验证、读取证据并给出可执行反馈的角色。
Rubric	评价量规	把主观或复杂质量拆成可评分、可讨论、可迭代的检查项。
Contract	契约	Generator 与 Evaluator 在编码前谈妥的 done-ness、测试和边界条件。
Trace	运行轨迹	Agent 的中间输出、工具调用、判断路径和失败记录，是调试 Harness 的主要材料。

若使用简单符号， P 表示 Planner， G 表示 Generator， E 表示 Evaluator， $C = \{c_1, c_2, \dots, c_n\}$ 表示双方谈妥的契约准则集合。符号只用于压缩结构，不用于把这场工程讲座改写成数学论文。



图 1：工作坊标题页给出主题与讲者：Ash Prabaker 和 Andrew Wilson 讨论如何让长程运行 Agent 保持主线。¹

1 为什么长程 Agent 会丢线

1.1 从 20 分钟到数小时：问题的尺度变化

Ash Prabaker 开场先把听众从常见演示里拉出来：许多公司会展示“让 Agent 一次性做出一个浏览器”这类结果，却很少讲清楚外层 Harness 到底做了什么。他给这场 workshop 定下的范围，是能连续运行 5 到 6 小时甚至更久的 Agent，而 Andrew Wilson 随后补上历史尺度：一年前的 Claude Code 连 bash 字符串转义都容易出错，单次运行大约 20 分钟；一年后，Claude Code 的大部分代码已经由 Claude Code 自己编写，并且可以持续运行数天（视频 00:00:31–00:02:24）。

长程运行 Agent（long-running agent）指能在数小时甚至更久的时间窗内持续推进任务的智能体。它的难点不在于“多调用几次模型”，而在于目标、约束、已完成状态、失败证据和下一步计划会随时间漂移。若把一次短对话看作一次短跑，长程 Agent 更接近一场接力赛：每一棒都要知道上一棒交出了什么、还欠什么、什么证据证明已经跑到这里。

这里的“丢线”有一个很具体的含义：Agent 还在运行，甚至还在输出看似合理的步骤，但它对任务目标、当前事实和完成标准的把握已经松了。一个按钮可以出现在界面上，模型便宣布“功能完成”；可后端路由、状态写入、错误处理和真实交互路径可能都不存在。Andrew 在 00:03:47–00:04:03 给出这个例子，说明长程任务里的“完成感”常常先于“完成证据”出现。

¹ 视频画面时间区间：00:00:42–00:00:47。若为视频帧截图，时间区间对应原视频画面。

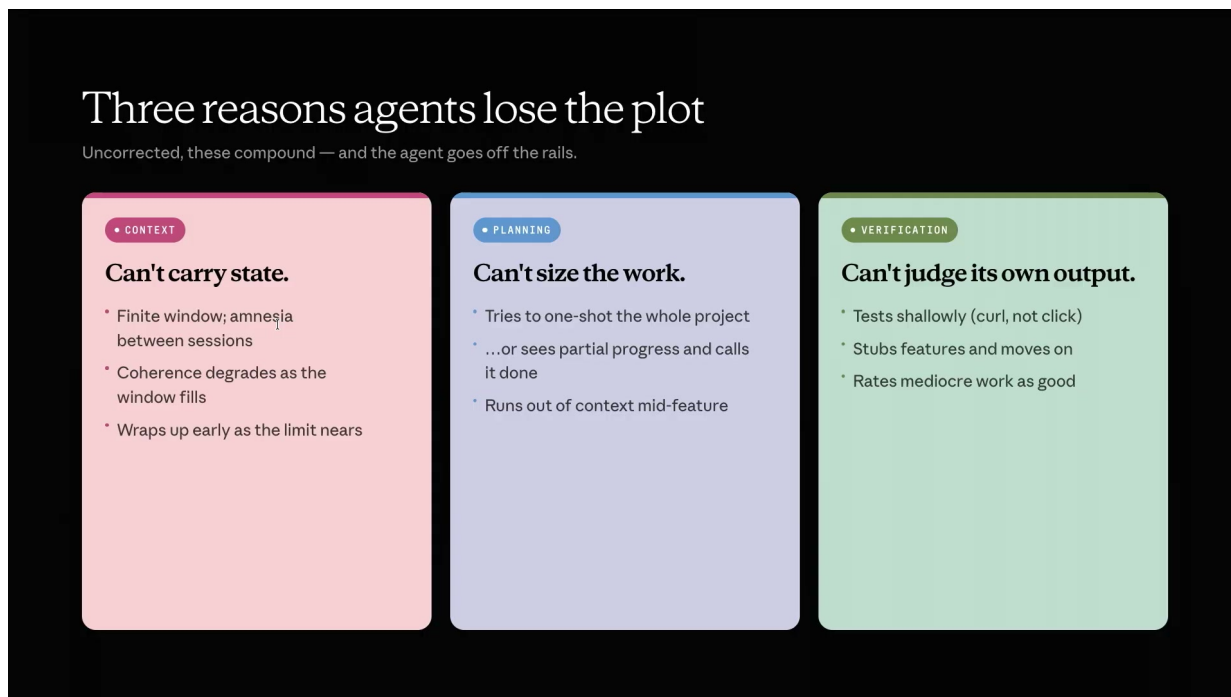


图 2：长程 Agent 失控的三类根因：上下文携带、工作规模判断和输出自评。后续 Harness 机制都在回应这三类失败。²

1.2 三类失控来源：上下文、规划、判断

Andrew 把长程 Agent 的失控来源分成三类：上下文、规划、自我判断。第一类是上下文问题。上下文窗口（context window）是模型一次推理时能直接看到的输入范围，包含系统提示、用户目标、文件片段、工具结果、历史消息等。窗口外的信息不会自动存在，需要依靠摘要、检索、文件、检查点或新会话重新带回。它像一张工作台：桌面上能摆多少材料有限，桌下的材料必须贴标签、建索引、按需拿回。

上下文窗口有限会造成“新会话失忆”：启动一个新 session 时，Agent 必须重新获得任务背景。更棘手的是上下文腐化（context rot）：会话越长，模型对目标、约束、局部事实和未完成事项的稳定性越低，输出仍流畅，内部指针却逐渐偏移。连贯性（coherence）就是目标、约束和执行状态在长时间内保持一致的能力；它考察模型能否沿着同一条工程证据链做判断，持续输出文本并不能证明连贯性仍在。

上下文腐化和上下文焦虑要分开看。上下文腐化发生在长会话本身：模型读过太多临时事实后，容易把旧目标、新假设和半成品混在一起。上下文焦虑（context anxiety）发生在接近窗口上限时：模型意识到空间快耗尽，可能过早收尾，把“还需要验证”的状态写成“已经完成”（视频 00:02:43–00:03:16）。

第二类是规划问题。Andrew 说模型开箱即用并不擅长长计划：它可能试图一次做完所有事，也可能实现半个功能后停住，或者把上下文用光后留下半成品应用（视频 00:03:16–00:03:35）。这里的失败机制很朴素：任务越长，隐含依赖越多；如果没有外部结构把大目标拆成可验证的小单元，模型会在“先做什么、何时停、如何证明完成”上不断消耗判断力。

²视频画面时间区间：00:02:30–00:04:15。若为视频帧截图，时间区间对应原视频画面。

第三类是自我判断问题。模型会讨好用户，也会讨好自己刚刚写出的代码。它看到一个半成品功能，倾向于接受表层信号：按钮存在、页面可打开、测试输出看起来绿了。真正的工程判断需要追问后端路径、状态变化、异常分支、权限、数据持久化和用户实际路径。长程 Agent 一旦把弱证据当成完成证据，后续步骤会把这个错误当作地基继续盖楼。

工具调用（tool calling）是模型选择并调用外部工具来读取、执行、测试或修改环境的能力。它让模型从“只会说”变成“能拿证据”：运行测试、打开浏览器、读文件、改代码、查看日志。可工具调用本身不保证判断正确；如果模型只调用了浅层工具，或者误读工具结果，长程运行仍会在证据链上断裂。

1.3 模型能力与 Harness 共同演化

Andrew 给出两个改进杠杆。第一个杠杆是模型权重本身：训练让模型更会规划、更会使用工具、更会处理上下文。视频里提到一个 meter chart：在最小脚手架下、完成 50% 任务这个口径里，Opus 3.7 大约能跑 1 小时；一年后的 Opus 4.6 大约能跑到 12 小时（视频 00:04:14–00:04:44）。这个数字不应读成“12 小时后万事大吉”，它说明模型底座已经把可持续运行的边界往外推了一大截。

Fix #1: Train the behavior into the weights.

Slow to ship — months per release. Free at runtime, works in every harness.

METR's task-completion time horizon: the human-time length of task a model completes with 50% reliability, in a minimal scaffold.

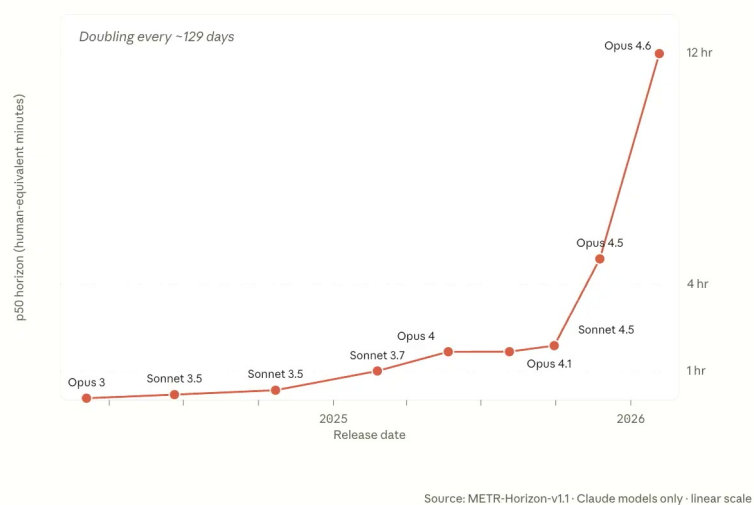


图 3：模型权重侧的进步：METR p50 任务时长曲线显示 Claude 系列模型可处理任务的时间跨度快速上移。³

³ 视频画面时间区间：00:04:20–00:04:50。若为视频帧截图，时间区间对应原视频画面。

Fix #2: Wrap the model in a harness.

Fast to ship — days. But adds cost, latency, and complexity to every run.

Claude Agent SDK — “the same agent loop, tools, and context management that power Claude Code.”

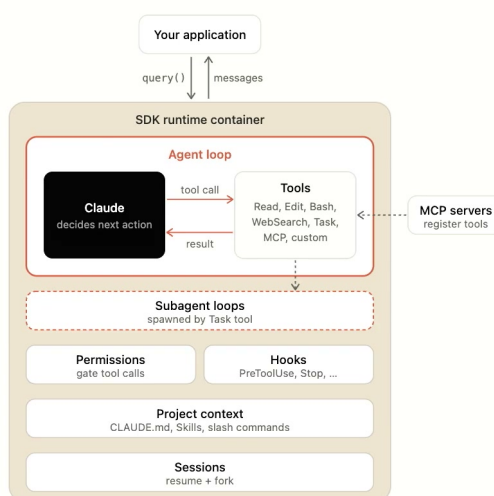


图 4: Agent SDK Harness 把模型放进工具循环、权限、hooks、项目上下文、MCP 服务和子 Agent 会话之中，形成长程运行的外部骨架。⁴

第二个杠杆是 Agent Harness，也就是包在模型外部的执行脚手架。Agent Harness / 智能体脚手架指外层结构：核心工具循环、权限系统、上下文注入、Claude.md、skills、slash commands、子 Agent、MCP 服务器、停止条件、状态文件和领域特定编排。模型像发动机，Harness 像变速箱、仪表盘、刹车、路线表和维修记录；发动机更强会让车跑得更远，外层结构决定它能不能沿着正确路线、安全地跑完。

长程能力来自模型能力与 Harness 的共同演化。模型决定单步判断、规划和工具选择的上限；Harness 把任务切成可恢复、可验证、可交接的外部结构。只升级模型，旧 Harness 可能留下多余摩擦；只堆 Harness，弱模型会在每个环节产生新的误判。

在 Andrew 的描述里，Agent SDK 提供的核心 loop 会让 Claude 模型决定下一步、选择工具、从 MCP 服务器拉工具、把任务交给子 Agent，并读取 Claude.md、skills、slash commands 等上下文，还要经过权限系统约束（视频 00:04:51–00:05:34）。这说明“让 Agent 跑更久”依赖模型、工具、状态、权限和角色分工组成的工程系统，单一 prompt 技巧撑不起这种时间尺度。

1.4 本章小结

本章先建立了问题的真实尺度：从 20 分钟到 5–6 小时再到数天，失败已经超出某一步代码写错的范围，进入上下文、规划和判断随时间漂移的范围。上下文窗口有限会带来失忆、腐化和焦虑；规划薄弱会让任务拆分和停止条件变形；自我判断薄弱会把按钮、页面或浅层测试误认成完整功能。

解决方向同样有两条线：模型权重继续提升，Harness 继续把任务外化成工具循环、权限、状态、子 Agent、skills 和可验证证据。后面的时间线章节会说明，这套脚手架来自 Claude Code 一年里逐步发布的原语，经过多轮产品与模型共同演化。

⁴视频画面时间区间：00:04:50–00:05:55。若为视频帧截图，时间区间对应原视频画面。

2 Claude Code 演化史：长程运行的原语从哪里来？

2.1 从 Artifact、Computer Use 到 MCP

Andrew 把 Claude Code 之前的阶段称为“prehistory”。Sonnet 3.5 和 Claude.ai 的 Artifacts 让模型第一次能看见自己构建出的东西，并围绕输出继续迭代；这是一种反馈回路的雏形。Computer Use 进一步让模型能点击界面、截图、测试自己的代码；MCP / Model Context Protocol 则提供了一层协议，让模型接入外部工具、资源或服务器（视频 00:05:58–00:06:34）。

MCP 可以理解为“给模型接外设的协议层”。没有工具协议时，模型主要依赖对话里的文本描述；有了 MCP，模型可以通过服务器获得文件系统、浏览器、数据库、设计工具、部署平台等能力。它把推理连接到真实环境，让模型能在外部系统里取证和行动。

这个阶段的关键是“看见输出后再改”的回路出现了。没有这个回路，Agent 像闭眼写代码；有了 artifacts、截图、工具和外部资源，它至少能把生成结果和环境反馈放到同一个循环里。后续所有长程 Harness 都在扩展这个基本结构：生成、观察、判断、修正。

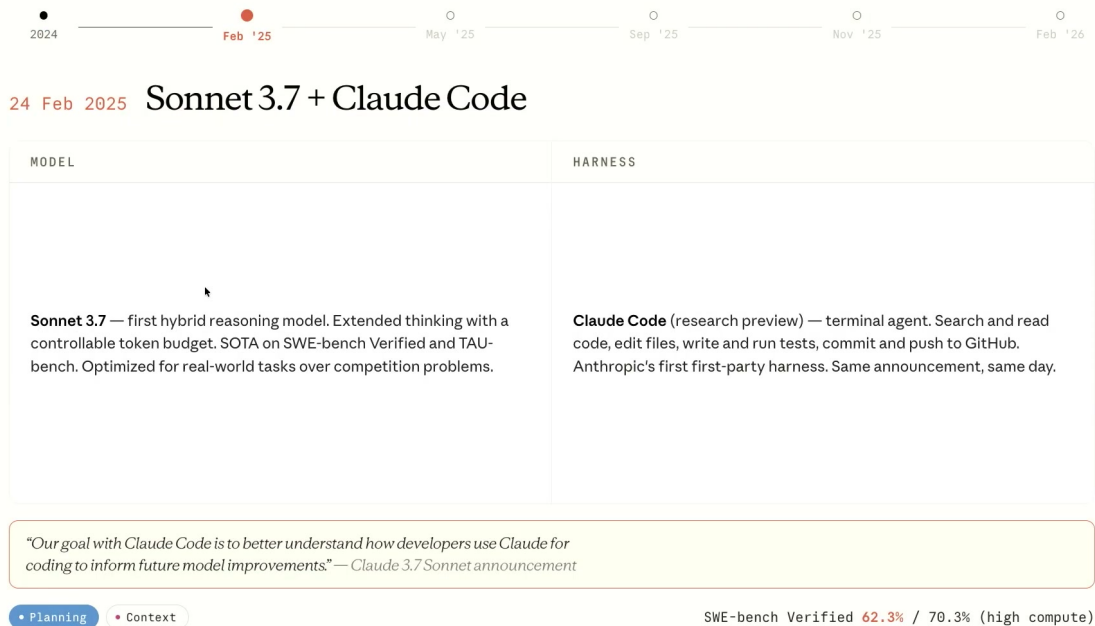


图 5: Sonnet 3.7 与 Claude Code 研究预览同期开启，说明模型能力与第一方 Harness 从早期就互相牵引。⁵

2.2 Claude Code：产品也是模型改进的实验场

2025 年 2 月，Sonnet 3.7 发布，Claude Code 以 research preview 形式出现。Andrew 特意引用了当时 release 里的目标：理解开发者如何使用 Claude 编码，并把这些信息反馈到未来模型改进中（视频 00:06:34–00:07:08）。这句话很重要，因为 Claude Code 同时承担两个角色：面向开发者的产品，以及观察模型失败模式、采集使用模式、推动模型训练的实验场。

到 2025 年 5 月前后，Opus 4 和 Sonnet 4 发布，Claude Code 进入 GA，并发布 Claude Code SDK。这个 SDK 可以看作当时驱动 Claude Code 的 Harness 被抽象成可复用开发接口：外部开发者不只拿

⁵ 视频画面时间区间：00:06:34–00:07:00。若为视频帧截图，时间区间对应原视频画面。

到一个聊天框，也开始拿到能包装工具循环、上下文、权限和任务执行的框架（视频 00:07:33–00:07:55）。

Agent SDK 是 Claude Code SDK 泛化后的智能体开发框架。它把原本服务于编码产品的 Harness 原语抽出来，让开发者能在编码之外的领域构建长程 Agent，例如网页应用生成、数据处理、内部运维或多角色协作流程。

这条产品线也给后文一个判断标准：当模型更强时，Harness 不会原地不动。旧脚手架里有些部分会被吸收到模型能力中，有些部分会变成平台原语，有些部分会因为新模型的短板改变而重新组合。

2.3 Ralph Loop：可预测失败的循环

Andrew 在 00:07:55–00:09:49 插入 Ralph Wiggum 技术，也就是常说的 Ralph Loop / Ralph 循环。它的基本做法是有一个 prompt 喂给 Claude Code CLI 或类似代理，让它循环运行，直到任务完成。这个说法过于简化；Andrew 强调 Ralph Loop 里还包含规划阶段：先把 prompt 拆成若干 features，再选取一个任务，开启新 session，用新上下文窗口推进。

Ralph Loop 指循环调用 Claude Code CLI 或类似代理，并配合计划、退出条件、新上下文窗口和任务拆分来推进任务的模式。它的工程价值在于可预测：与其偶然成功、失败时不可复现，不如让系统以固定方式失败，便于人类定位边界和补 Harness。

Claude Code 后来也发布了自己的 Ralph Loop 插件。它和经典 Ralph Loop 有一个关键差异：它在单个 Claude Code session 内运行，依赖 compaction 随时间发生，没有每轮强制创建新上下文窗口。用户可以设置最大迭代次数、safe word，stop hook 会拦截 Claude 原本要停止的时刻；若退出条件没有满足，它会继续跑到命中某个 exit criteria（视频 00:09:12–00:09:49）。

Ralph Loop 的局限在于它主要延长执行，不自动提供强对抗判断。循环能让 Agent 多试几轮，却不能保证它会怀疑自己的半成品。如果缺少独立评估者，一个错误架构可能被连续修补，最后变成更难拆的半成品。

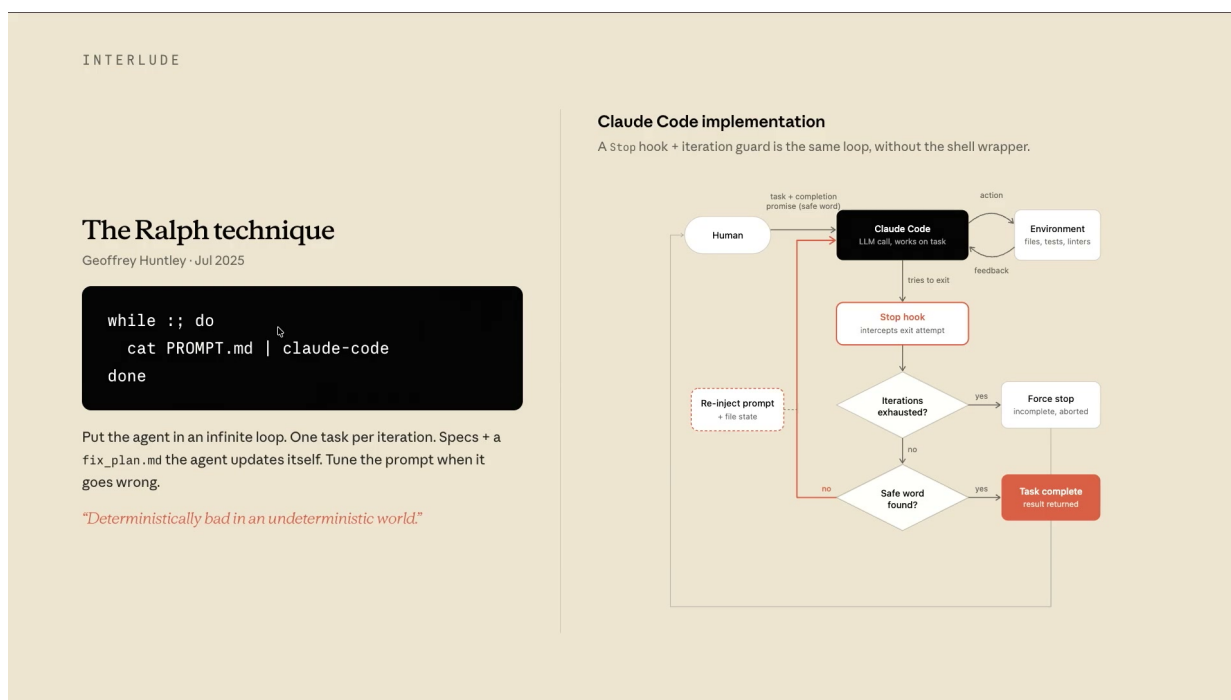


图 6: Ralph Loop 把 Claude Code 放入可重复运行的循环，用 stop hook、safe word 和迭代上限约束退出行为。⁶

2.4 Checkpoints、Sub-agents 与 Agent SDK

Sonnet 4.5 阶段，模型开始更会处理自身上下文：它能跟踪 token 消耗，并在接近窗口末端时管理上下文（视频 00:09:49–00:10:11）。Claude Code 2.0 同期引入 checkpoints。检查点（checkpoint）是可回退、可恢复的任务状态保存点；普通日志记录发生过什么，检查点还要让系统能回到某个较好的状态继续工作。

随后 Claude Code SDK 被重命名为 Agent SDK，因为 Anthropic 意识到这套能力已经超出编码场景。Andrew 说，虽然当时讲了很多 coding，但长程 Harness 可以应用到其他领域（视频 00:10:22–00:10:40）。这一阶段还有一个硬数字：Sonnet 4.5 配合当时的系统，已经能运行约 30 小时（视频 00:10:40–00:10:49）。

Opus 4.5 与 Haiku 4.5 补齐后，运行多个 sub-agents 变得更经济。子 Agent（sub-agent / 子智能体）是由主流程派出去承担特定角色或任务的智能体，例如规划、执行、评估、查资料或跑测试。Andrew 给出的典型分工是：Opus 4.5 负责规划，Sonnet 4.5 作为 workhorse 执行代码（视频 00:10:52–00:11:14）。

Skills 的 progressive disclosure 解决的是上下文预算问题：先只加载技能的 front matter，让模型知道“有这个工具箱”；真正触发时，再加载正文、参考材料或可执行代码。它像索引卡片，先告诉模型书架上有什么，等需要时再把整本书拿下来。

同一阶段还出现了 programmatic tool calling。程序化工具调用指模型临时写代码来组织一串工具操作，只把最终结果带回上下文。普通工具调用像每次都把原始票据贴进对话；程序化工具调用像先在本地把票据整理成报表，再把报表交给模型。这样可以减少上下文负担，尤其适合长程任务里反复扫描文件、汇总状态、批量验证的场景（视频 00:11:44–00:12:05）。

⁶视频画面时间区间：00:07:55–00:09:49。若为视频帧截图，时间区间对应原视频画面。

2.5 Agent Teams 与 Server-side Compaction

2025 年 11 月前后, Anthropic 发布第一篇长程 Agent blog。Andrew 用“写一个浏览器”“做一个 Slack clone”“做一个 Salesforce clone”这类模糊人类提示来解释第一代长程 Harness: initializer agent 先把简单 prompt 分解为持久化工件 (persistent artifacts), 例如 `featurelist.json`、`progress` 文件、Git repo、init script, 以及每个 feature 是否完成、测试是否通过的状态标记 (视频 00:12:05–00:13:12)。

持久化工件是跨会话保留的计划、状态、测试结果、文件、截图或结构化证据。它承担“外部记忆”的角色。长程 Agent 不能只依赖模型内部记忆, 因为新 session 会失忆, 长 session 会腐化; 持久化工件把关键状态放到文件系统或其他可读取介质中。

他们选择 `featurelist.json` 还有一个很工程化的原因: 团队发现模型更容易随手覆盖 Markdown 文件, 对 JSON 这类结构化状态文件则更谨慎 (视频 00:12:48–00:12:58)。所以长程 Harness 的状态文件并非只要“写下来”就够, 还要选一种能降低误覆盖概率的格式。JSON 像带字段名的账本, 模型更容易把它当作状态表维护; Markdown 像自由文本白板, 模型更容易把整段重写成一份新的说明。

这个 Harness loop 的动作顺序很具体: 在新上下文窗口中读取当前目录和 `progress` 文件, 运行 init script 或 smoke test 启动服务器, 选一个尚未通过所有测试的 feature, 实现它, 用 Puppeteer 做验证; 若通过, 则写 Git commit 并把该 feature 状态改为 passes; 若还有未完成 features, 就在新的上下文窗口继续下一轮 (视频 00:13:12–00:14:14)。这就是“长程”从口号变成流程的时刻。

到 Opus 4.6 / Sonnet 4.6 阶段, Sonnet 4.6 以更低成本接近 Opus 级能力, 成为 Claude Code 的执行 workhorse; Opus 4.6 则更擅长规划、选择工具和长时间运行。Andrew 再次引用 meter chart: 从约 4 小时跳到 12 小时, 说明 Opus 4.6 的 agentic 能力显著增强 (视频 00:14:20–00:15:03)。

Agent Team / Agent 团队指多个子 Agent 能互相协作、分工和通信的框架。它比“主 Agent 派任务、所有结果回主 Agent”更进一步: 子 Agent 之间可以彼此交流、协调局部决策, 只在需要时向主 Agent 汇报。这个通信拓扑变化很关键: 主 Agent 从“所有消息的中转站”变成“只接收关键汇报的编排者”, 主上下文不必承载每个子任务的来回细节, 团队结构也更接近真实工程小组 (视频 00:15:03–00:15:30)。

Server-side compaction / 服务端上下文压缩指服务器侧把长会话或中间状态压成摘要, 再让模型继续运行。Andrew 说, 这使模型可以“indefinitely”地跑下去, 并且压缩发生在服务端 (视频 00:15:30–00:15:40)。但这个定义必须带着限制理解: 压缩会丢信息, 能延长会话长度, 不等同于自动保持连贯性。第 6 章和总结会继续强调, compaction 与 coherence 之间没有等号。

Andrew 还给了一个比 METR 曲线更贴近日常实验的经验数: 到 Opus 4.6 阶段, 许多完整应用已经不需要 30 小时长跑, 通常约 3 到 5 小时就能生成一个开箱可运行、功能较完整的应用 (视频 00:16:14–00:16:35)。这个数字不能当成 SLA, 它的价值在校准尺度: 长程 Agent 的实际时间单位已经从“整天守着”等级, 压到“半个工作日回来验收”的等级。

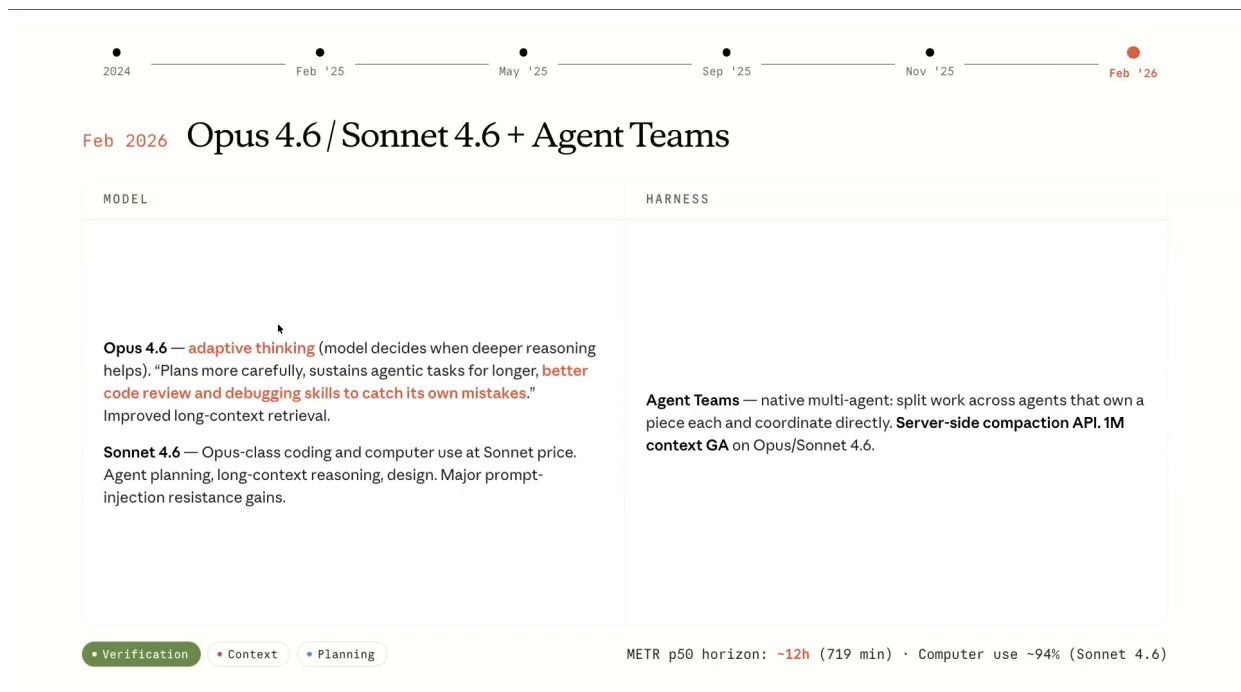


图 7: Opus 4.6 与 Sonnet 4.6 阶段, Agent Teams、server-side compaction 和 1M context 成为新的长程运行原语。⁷

Every release, agents run longer unattended

Beat	Model	Harness shipped with it	Primary attack	METR p50 horizon ¹	With harness ²
Feb '25	Sonnet 3.7	Claude Code	Planning	~1h (60 min)	"tasks that would normally take 45+ min, in a single pass"
May '25	Opus 4 / Sonnet 4	CC GA, Code SDK, code-exec, Files API	All three	~1h 40m (100 min)	Rakuten: 7h autonomous refactor
Sep '25	Sonnet 4.5	CC 2.0, checkpoints, memory tool, Agent SDK	Context	~1h 53m (113 min)	30+ h sustained focus on multi-step tasks
Nov '25	Opus 4.5 / Haiku 4.5	Skills, prog, tool calling, tool search, compaction	Context · Planning	~4h 53m (293 min)	Multi-day claude.ai clone build (200-feature spec)
Feb '26	Opus 4.6 / Sonnet 4.6	Agent Teams, server compaction, 1M ctx	Verification	~12h (719 min)	6h retro game maker / 4h DAW, end-to-end

¹ METR-Horizon-v1.1, minimal scaffold, 50% success (corrected Mar 3 '26). ² Anecdotal, from Anthropic announcements & engineering blogs.

图 8: 发布节奏表把模型、Harness 原语、主要故障面和无人值守时长放在同一坐标系里, 便于观察共同演化。⁸

2.6 本章小结

Claude Code 的长程能力来自一串逐步发布的原语: Artifacts 让模型看见输出, Computer Use 让模型触碰界面, MCP 接入外部工具, Claude Code research preview 把真实开发者使用反馈送回模型

⁷ 视频画面时间区间: 00:14:20-00:15:00。若为视频帧截图, 时间区间对应原视频画面。

⁸ 视频画面时间区间: 00:16:05-00:16:35。若为视频帧截图, 时间区间对应原视频画面。

改进，Claude Code SDK / Agent SDK 把 Harness 抽象成开发框架，Ralph Loop 把循环和退出条件工程化，checkpoints 让状态可回退，sub-agents 让角色分工变便宜，programmatic tool calling 降低上下文负担，Agent Teams 让多 Agent 能彼此协调，server-side compaction 延长连续会话。

这条时间线给后续章节一个底层判断：长程 Agent 依赖模型能力、工具协议、上下文管理、持久化工件、验证循环和多角色协作共同堆叠，单个神奇 prompt 承担不了这套工程职责。Andrew 的结论很直接：模型变强后，Harness 不会消失，它会随着模型短板一起演化；某些旧结构会被删掉，某些新结构会变成下一轮能力的训练材料。

3 当前 Harness 范式：用角色分离压住自评偏差

Ash 在 00:17:31 先问现场：现在有多少人的 Agent 正在后台干活？只有零星几个人举手。他随后把 Andrew 的历史线索收束到一个更工程化的问题：如果 frontier 会移动，开发者今天能复制的 Harness 模式是什么？这一段的回答很直接：把“做东西的人”和“挑毛病的人”拆成两个角色、两个上下文窗口、两套系统提示和两种工具行为。

3.1 从自评到对抗式评价

本章第一次使用三个核心角色。生成器 / Builder 指负责实现功能、写代码或构建产品的角色；Evaluator / 评估器，也可称 Critic，指负责检查输出、真实使用产品、运行测试并给出可执行反馈的角色；对抗式评估器指与生成器职责隔离、目标上保持压力的评估器，它像 GAN 里的 discriminator，但落点是工程产品的质量判断。

Ash 在 00:18:42 明说这个想法“偷”自 GAN。经典 GAN 的直觉可以写成一个很小的博弈：

$$G(z) \rightarrow \hat{x}, \quad D(x) \rightarrow [0, 1]$$

其中：

- G 表示 generator，输入噪声或条件 z ，输出一个候选样本 $\hat{x}$ 。
- D 表示 discriminator，输入样本 x ，输出它看起来像真实样本的概率。
- 在本讲的工程类比里， G 不再生成图片样本，而在这里是生成应用、页面或代码； D 也不再训练神经网络判别真假，而在这里是作为 Evaluator 对产品进行使用、评分和批评。

这个类比的关键不在神经网络训练细节，而在压力结构。00:19:01 到 00:19:12，Ash 把它压缩成一句机制：generator builds, evaluator grades。生成器的任务是把产品做出来，评估器的任务是把产品按证据拆开看。两者共享最终目标，但不共享同一段自我辩护的上下文。长程 Agent 最怕“我刚刚做了很多，所以我倾向于相信自己做完了”；角色隔离就是把这种心理惯性切断。

这里的“对抗”也不等于互相捣乱。更准确地说，评估器有独立的失败预算：它可以说页面只是看起来完成，可以指出测试太弱，可以要求重跑，还可以把低质量结果判为不值得继续修补。人类团队里，做菜和品菜也是两种能力；Ash 在 00:20:16 用艺术品和餐食作类比，批评一顿饭比亲手做出同等水平的饭更容易。LLM 也有类似差距：把一个独立 Critic 调得严厉，比让同一个 Builder 在兴奋地建造时保持冷酷自审更可控。



图 9: 生成器与评估器被拆成两个角色后, Builder 负责产出, Evaluator 负责施加外部判断压力。⁹

自评循环的危险在于“同一份上下文”会持续奖励已有判断。生成器刚写完某个交互, 就容易把按钮、页面和局部测试当作完成证据; 独立评估器拿到的是产品和契约, 它的默认动作是验证, 而非替生成器解释。

3.2 Evaluator 必须真实使用产品

00:19:12 到 00:19:25 的细节很重要: Evaluator 不能只读 diff。Ash 说评估器会用 Playwright 打开 live pages, 点击页面, 尝试流程, 再把 critique 交还给 generator。这个要求把评估器从“代码审稿人”变成“产品使用者”。差别很硬: 读 diff 能看到函数是否改了, 打开页面才能看到按钮是否可点、路由是否通、表单是否提交、游戏是否真的响应键盘。

在 Web 应用里, Playwright 的价值接近一台不会走神的验收机器。它可以执行这样的证据链: 启动页面、定位按钮、触发点击、等待 DOM 变化、读取截图、检查控制台错误。用集合语言说, 评估器的重点从单个文件变化 Δf 移到用户可观察行为集合 B :

$$B = \{b_1, b_2, \dots, b_n\}$$

其中:

- B 表示产品暴露给用户的行为集合。
- b_i 表示一个可观察行为, 例如“点击新建项目后出现对话框”“方向键让玩家移动”“表单提交后出现成功状态”。
- Evaluator 的任务是采样并验证这些 b_i , 再把失败项写成反馈。

⁹视频画面时间区间: 00:17:28–00:19:40。若为视频帧截图, 时间区间对应原视频画面。

这也是为什么 Harness 要拆 system prompt 和 job。生成器的系统提示应该鼓励它推进实现、保持创造力、修复错误；评估器的系统提示应该鼓励它挑剔、复现、打分、拒绝空话。两个角色如果共用同一个上下文窗口，评估器很容易读到生成器的意图说明，然后把“它想这么做”误判成“用户已经能这么用”。

“评估器真实使用产品”是长程 Agent 的证据层。单元测试、静态检查和 diff 审查仍然有价值，但它们经常缺少 UI 层事实：动画是否卡住、焦点是否落错、空状态是否可理解、用户是否能从入口走到完成。Playwright MCP 或浏览器工具把这些事实带回 Agent 回路。

3.3 把品味写成 Rubric

Rubric / 评价量规指把主观或复杂质量拆成可评分、可讨论、可迭代的标准。设计品味指对界面美感、层次、原创性、工艺感和适用性的判断能力。AI slop 可以译为 AI 味模板化输出，指模型常见的廉价审美或低密度产物，例如泛紫色渐变、装饰堆砌、同质化卡片和缺少产品语境的视觉噪声。

Ash 在 00:20:43 转入 Critic 设计，问题从“它能不能运行”升级为“它看起来和用起来是否好”。全栈应用的质量有大量模糊区域：页面是否有层次、交互是否自然、功能是否像一个真实产品、视觉是否有原创性。很多人会说品味不可评分，Ash 的处理方式更朴素也更尖锐：只要团队有足够明确的判断，就把判断写下来。

00:21:41 到 00:21:48，他给出四个 Rubric 维度：design、originality、craft、functionality。可以把一次评估写成加权分数：

$$S = w_d s_d + w_o s_o + w_c s_c + w_f s_f$$

其中：

- S 表示总评分，只用于辅助排序和触发反馈，不能替代文字 critique。
- s_d, s_o, s_c, s_f 分别表示设计、原创性、工艺感、功能性的单项评分。
- w_d, w_o, w_c, w_f 是四项权重，且通常满足 $w_d + w_o + w_c + w_f = 1$ 。

在 00:21:48 到 00:22:05，Ash 说他们当前会把权重偏向 design 和 originality，因为 Opus 4.6 的 functionality 已经相对强。这个取舍很现实：如果模型已经能把基础功能补上，评估压力就应该转向它仍然容易偷懒的地方，比如紫色渐变、模板感、缺少产品身份、所有页面都像同一个 SaaS demo。

Rubric 还需要校准。00:22:14 到 00:22:23 提到 few-shot reference examples：给评估器看一些参考站点，让它的 taste 收敛到团队的 taste。这里的 few-shot 不只是“举几个例子”，更像给评估器装一把尺。没有参考例，评估器可能把花哨当精致；有了正反例，它才能区分“视觉密度高”和“装饰堆砌”，“有原创性”和“随机换颜色”。

Making subjective quality gradable

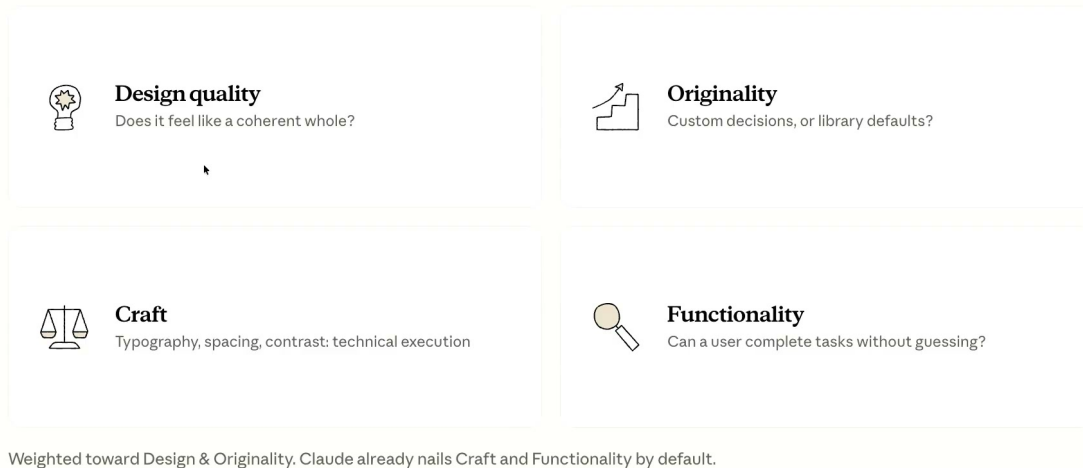


图 10: 主观质量通过 Rubric 被拆成设计质量、原创性、工艺和功能性，评价才有可讨论、可迭代的抓手。¹⁰

The frontend loop in action

Prompt: "Create a website for a Dutch art museum."

5-15 iterations per run. Evaluator navigates the live page with Playwright, scores it, sends a critique. Generator decides: refine, or pivot entirely.

Full runs stretch to ~4 hours.

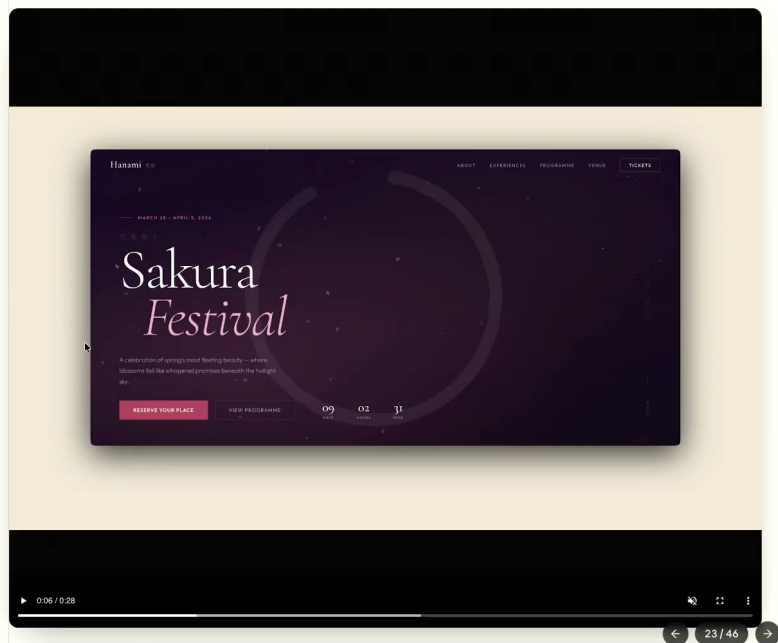


图 11: 前端设计循环展示 Evaluator 如何真实打开页面、评分并反馈，使 Generator 在多轮迭代中改进产品质感。¹¹

3.4 重启比补丁更便宜的时刻

00:22:36 到 00:22:50, Ash 描述了一个 HTML/CSS 示例循环: generator 产出页面, evaluator 启动 Playwright, 导航、截图、按四个标准打分, 写 critique, 再交回 generator。这个循环可能跑 4 小

¹⁰ 视频画面时间区间: 00:21:30-00:22:25。若为视频帧截图, 时间区间对应原视频画面。

¹¹ 视频画面时间区间: 00:22:32-00:23:35。若为视频帧截图, 时间区间对应原视频画面。

时，经历 5 到 15 轮。真正有价值的部分出现在 00:23:07 之后：如果生成器在 originality 上持续低分，GAN 风格 Harness 会把整个结果丢掉，从头再试。

这对长程运行很关键。单次生成或 Ralph Loop 风格的循环常常沿着同一座坏房子继续装修：按钮再调一点、颜色再换一点、布局再挪一点，但底层产品结构已经偏了。对抗式评估器有权判断“继续贴补丁的边际收益太低”。工程上这叫止损，设计上这叫重新构图，Agent Harness 里它表现为重启一次生成路径。

如果把每轮产物记为 a_t ，评估器给出的 critique 记为 q_t ，普通补丁循环大致是：

$$a_{t+1} = \text{patch}(a_t, q_t)$$

而带重启权的 Harness 还允许：

$$a_{t+1} = \text{restart}(q_1, q_2, \dots, q_t)$$

其中：

- a_t 表示第 t 轮的产品状态。
- q_t 表示第 t 轮评估器的批评和证据。
- patch 表示在现有结构上修补。
- restart 表示保留学到的失败原因，重新生成产品结构。

这并不保证每次重启都更好。它只是给 Harness 一个普通自评循环缺少的动作空间：当评估信号显示方向错了，系统可以换路线。长程 Agent 的质量差异，经常就藏在这类动作空间里。

3.5 本章小结

本章的核心机制可以压成四句话。第一，生成器负责建造，评估器负责验证，两者要有独立上下文、独立系统提示和独立工具行为。第二，Evaluator 要真实使用产品，Playwright 点击、截图、导航和流程验证比只读 diff 更接近用户事实。第三，Rubric 把设计品味、原创性、工艺感和功能性写成可评分标准，再用参考例校准，才能压住 AI 味模板化输出。第四，对抗式 Harness 的价值不止在多轮修补，也在它能识别补丁无效的时刻并要求重启。

4 Planner 与 Contract：把模糊愿望压成可测试断言

在 00:23:51，Ash 从“漂亮页面”转向“完整可工作的应用”：他们加入了第三个角色 Planner。这里的重点不在多一个 Agent 数量，而在职责边界。Planner 只给高层方向，Generator 与 Evaluator 在写代码前谈清楚 Done 的定义，随后用文件系统把约定保存下来。这个小动作把一行愿望压成了可执行的工程契约。

4.1 Planner 只画外轮廓

Planner / 规划器指将一句提示转成高层方向、外轮廓和 sprint 框架的角色。它不应该提前规定所有技术细节；它的职责更像产品经理给出方向和阶段，无需替每个实现者写完测试步骤。

00:24:03 到 00:24:12，Ash 说 Planner 会把 one-line prompt 拆成 deliberately high-level spec。随后他立刻补充，Planner 只把 general workflow 拆成一系列 sprints，不去规划产品的 granular technical

details。这个限制很重要，因为长程运行会放大早期错误。00:24:34 到 00:24:43 的说法很直白：Planner 仍然可能犯错，一旦它在细节上犯错，错误会穿过每个 sprint，在多小时周期里被放大。

可以把三角色写成一个轻量流程：

$$P \rightarrow (G \leftrightarrow E \text{ on } C) \rightarrow A$$

其中：

- P 表示 Planner，负责高层产品方向和 sprint 外轮廓。
- G 表示 Generator，负责实现。
- E 表示 Evaluator，负责验证和批评。
- C 表示 Generator 与 Evaluator 协商出的 Contract。
- A 表示最终应用、页面、代码或其他可检查工件。

00:24:46 到 00:25:02，Ash 把这个结构类比成 PM、IC、QA。这个类比很朴素，也很有杀伤力：他们没有发明一种全新的组织形态，只是给每个角色一个自己的上下文窗口。上下文隔离让 PM 式的高层方向、IC 式的实现冲动、QA 式的挑剔验证不会混成一锅。混在一起时，Agent 容易在同一段对话里先许愿、再实现、再替自己验收；拆开之后，每个角色都要通过工件和证据说话。

Scaling to full-stack: three agents



图 12: Planner、Generator、Evaluator 三角色分工：Planner 压出规格，Generator 按 sprint 生成，Evaluator 用 Playwright 与 rubric 反馈 pass/fail。¹²

¹²视频画面时间区间：00:23:44–00:25:04。若为视频帧截图，时间区间对应原视频画面。

4.2 Generator 与 Evaluator 先谈 Done 的定义

User story / 用户故事指用户希望完成的目标叙述，例如“作为玩家，我想创建一座城堡并让角色移动”。Done-ness 指一个功能可以被判定完成的可观察条件。Contract / 契约指 Generator 与 Evaluator 在编码前协商出的 done-ness、测试和边界条件。

本段最值得记住的时间点是 00:25:14：在 generator 写下第一行代码之前，两个 Agent 会先协商 done 到底是什么意思。Ash 举的对话结构是：Generator 提议“我会构建 X 功能，你应该通过测试 Y 来验证”；Evaluator 推回来范围太大、测试太弱、还漏了 XYZ edge case。这个来回发生在磁盘文件里：一个写 Markdown，另一个读 Markdown，反复迭代，直到双方同意。

这一步是长程 Harness 的硬约束。先谈 Contract，后写代码。原因很简单：写完再定义 Done，生成器会天然偏向把已有实现解释成完成；写之前定义 Done，评估器可以在实现还没有沉没成本时把标准钉住。这里的 Contract 源自用户原始 spec，并进一步变成用户故事和可测试断言之间的桥：

$$\text{user story} \rightarrow C = \{c_1, c_2, \dots, c_n\}$$

其中：

- user story 表示用户或 Planner 给出的目标叙述。
- C 表示契约集合。
- c_i 表示一个可检查断言，例如“按方向键时玩家坐标变化”“角色不能穿过城堡墙”“AI 助手生成的关卡会进入 play mode”。

00:25:59 到 00:26:09，Ash 说 Evaluator 后续会按照两个 Agent 自己决定的 contract 打分，Planner 开头一次性写出的原始 spec 只提供起点。这个设计让 Planner 不必过度细化，又不会让实现阶段失去验收标准。Ralph Loop 的固定 `plan.md` 风格循环缺的正是这一层争论：主循环按计划继续跑，但另一侧没有独立角色在编码前质疑计划是否足够可测。

弱 Contract 会制造弱反馈。比如“游戏应该好玩”只能带来泛泛 critique；“方向键移动、碰撞墙体、生命值和分数随事件变化、debug HUD 显示实时坐标”会迫使 Evaluator 打开应用、操作键盘、观察状态变化。

4.3 文件系统 Hand-off：低技术但可靠的胶水

Hand-off / 交接指角色间通过文件、摘要、工件或消息传递状态与约束。在本章场景中，Hand-off 具体指 Generator 和 Evaluator 通过磁盘上的 Markdown 契约、反馈文件和验收记录来交换信息。

Ash 在 00:25:43 到 00:25:52 说得很具体：back and forth via files on disk，一个写 Markdown，另一个读它。这听起来低技术，但它恰好适合长程 Agent。文件有三个好处。第一，它能被不同上下文窗口读取，避免把所有过程塞进同一段对话。第二，它能留下状态，失败后可以回看谁同意了哪些标准。第三，它能让评估器围绕固定文本追责，减少对生成器临场解释的依赖。

一个最小可用的磁盘交接可以长这样：

```
1 contracts/  
2   sprint_03_proposal.md      # Generator: will build X, verify with Y
```

```

3  sprint_03_evaluator.md      # Evaluator: scope pushback, weak tests, edge cases
4  sprint_03_contract.md      # Agreed done-ness and test assertions
5  sprint_03_result.md        # Evaluator verdict after using the product

```

Listing 1: Generator 与 Evaluator 编码前协商 Contract 的文件交接示意

这段示意里的关键文件是 `sprint_03_contract.md`。它是双方同意后的冻结点。后续 Generator 可以继续实现, Evaluator 可以据此验收, Integrator 或人类也能读到本轮到底承诺了什么。它还给 trace debugging 留下锚点: 如果结果失败, 读者能分清是 Planner 的方向错、Contract 太弱、Generator 没做到, 还是 Evaluator 没测到。

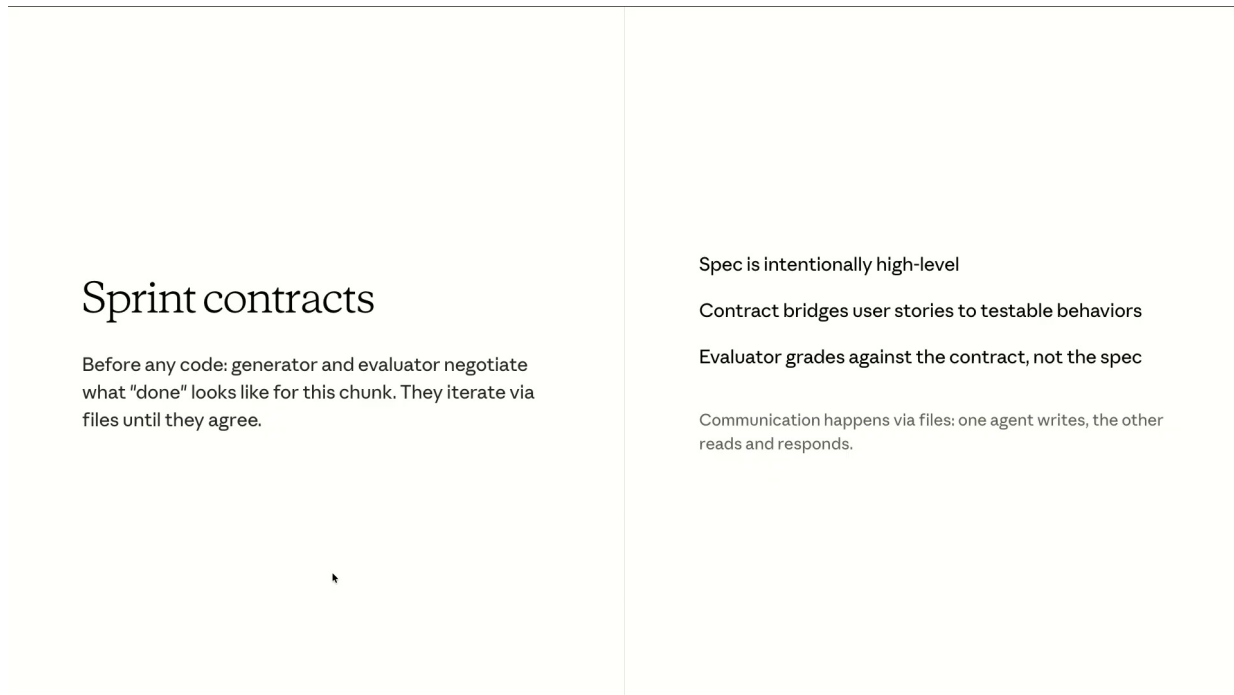


图 13: Sprint contract 把高层 user story 转成可验证行为; Generator 与 Evaluator 先通过文件谈妥 done, 再进入实现。¹³

4.4 游戏示例：从一句 AI Features 到可玩的系统

00:26:50 开始, Ash 给出一个对照实验: 同一句 prompt, “build a retro game maker”。他提醒现场, 这种跑法目前耗时很长也很贵; 00:29:02 说 Harness 版本大约花了 200 美元、6 小时。这个成本说明它尚未适合日常小需求的默认解法, 但示例清楚揭示了脚手架带来的能力差异。

先看没有 Harness 的单 Agent 循环。00:27:39 到 00:28:13, 开屏和 sprite editor 看起来还可以: canvas 在, palette 在, frame timeline 和 live preview 也在, 只是有点挤, color picker 只有黑色 swatches。表面上它理解了“复古游戏制作器”。真正断裂发生在 00:28:20 到 00:28:40: play mode 里有实体、分数、生命值, 可按方向键没有反应, 按空格也没有反应。它做出了游戏编辑器的外壳, 却没有理解“玩一个游戏并成功”需要哪些可观察行为。

Harness 版本的差异从产品命名就开始。00:29:07 到 00:29:18, 它把应用命名为 Retro Forge, 创建了新项目对话框和更完整的 canvas。Ash 明确说这些都不在原 prompt 里, 是 Planner 做出的产品决策。00:29:33 到 00:29:46, sprite editor 有 54 色 palette, 项目对话框里的 8-bit preset 会流入编辑器, sprite 以真实游戏比例显示。这里体现的是产品状态贯通, 控件数量增加只是表层现象。

¹³ 视频画面时间区间: 00:25:04-00:26:40。若为视频帧截图, 时间区间对应原视频画面。

最有代表性的例子是“AI features”。00:29:58 到 00:30:09, Planner 只写了一个很模糊的 AI features。Harness 把它变成应用内部的 AI-level assistant。用户可以输入类似“创建一座城堡, 让 sprites 守卫它”的请求。没有 Planner, 这句话甚至不会变成待处理 work item; 有 Contract negotiation, Generator 和 Evaluator 会把它压成一组检查: 助手能否接收自然语言、生成关卡元素、把结果应用到地图、进入 play mode 后仍可交互。

最后是可玩性证据。00:30:37 到 00:31:08, play mode 顶部左侧有 debug HUD, Ash 说这显然是为了让 evaluator 更容易验证。数字是实时的, physics loop 在运行, 方向键让玩家移动, 玩家会与城堡墙发生碰撞。Evaluator 真实启动游戏并尝试玩, 才知道这些功能需要被测试。这个示例把“Done 的定义”落到了键盘、坐标、碰撞、墙体和 HUD 上。

Example: retro game maker, solo vs full harness

	Solo agent	Full harness
Duration	20 minutes	6 hours
Cost	\$9	\$200
Play mode works?	No (entity wiring broken)	Yes
Sprite editor	Basic	Full palette, zoom, AI-assisted
Features in spec	4 (from prompt)	16 across 10 sprints

图 14: 同一条 retro game maker prompt 在 solo agent 与 full harness 下的结果对比: 时间和成本上升, 交互完整性也明显上升。¹⁴

¹⁴视频画面时间区间: 00:27:00-00:31:20。若为视频帧截图, 时间区间对应原视频画面。

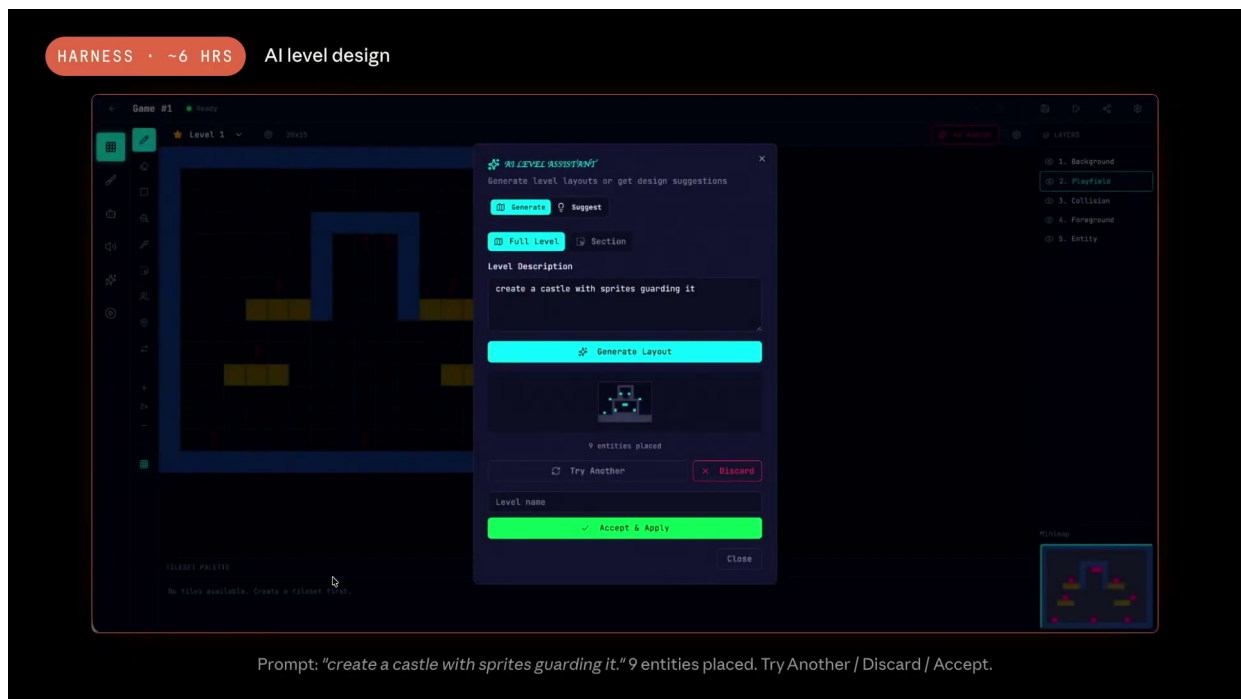


图 15: AI Level Assistant 把自然语言关卡需求转成可预览、可重试、可接受的关卡生成流程。¹⁵



图 16: Play mode 的 debug HUD 暴露位置、速度、状态和落地标志，让 Evaluator 能判断物理循环和碰撞边界是否真实工作。¹⁶

这个对照实验的结论很硬：同一个 prompt、同一个模型，弱输出和强输出的差别主要来自 scaffolding。Prompt 可以启动任务，Harness 决定任务如何被拆分、验收、修正和保留证据。

¹⁵ 视频画面时间区间：00:27:00-00:31:20。若为视频帧截图，时间区间对应原视频画面。

¹⁶ 视频画面时间区间：00:27:00-00:31:20。若为视频帧截图，时间区间对应原视频画面。

4.5 本章小结

本章给出长程 Harness 的第二个核心机制。Planner 只画高层外轮廓，避免把早期细节错误扩散到每个 sprint。Generator 与 Evaluator 在编码前协商 Contract：生成器提出建什么和怎样验，评估器压缩范围、加严测试、补齐边界条件，双方把同意的 Done-ness 写进磁盘文件。Hand-off 不需要花哨，普通 Markdown 文件已经能承担跨上下文交接。Retro Forge 示例说明，真正的完成要落到可观察行为：方向键、物理循环、碰撞、实时 HUD 和 AI 助手生成的关卡是否进入可玩系统。

5 可执行的具体性：27 条 Contract 与 Trace Debugging

上一章已经把 Planner、Generator、Evaluator 和 Contract 的角色关系讲清楚了。Ash 在 00:31:28 之后补上的关键工程细节是：这个系统能工作，核心在于把检查标准细到足够改变下一轮行为，而非只写一句“请严格检查”的大口号。具体性在这里属于控制信号的分辨率问题。信号太粗，Generator 只会收到“做得更好一点”这种无处落脚的反馈；信号足够细，它才知道该改哪条路由、哪个键盘事件、哪个 UI 状态和哪一段判断。

5.1 CI 没抓到的错误：Evaluator 可以抓到

00:31:28–00:31:53 这一小段里，Ash 举了几个很普通也很致命的例子：FastAPI 的 route ordering、delete key 的 Boolean logic bug、以及“每个 unit test 都过了，production 仍然会坏”的情况。这些问题听起来没有模型魔法，恰好说明 Evaluator 的价值。它在浏览器或运行环境里真的使用应用，所以它能碰到 CI 没覆盖到的路径。

FastAPI route ordering 的典型风险是动态路由和静态路由的匹配顺序互相遮挡。举个直观类比：如果门口先放了一个“所有穿蓝衣服的人都进这个房间”的牌子，再放“张三进另一个房间”的牌子，张三穿蓝衣服时就可能被提前拦走。Web 路由也是同一类问题：更宽泛的 pattern 如果先匹配，后面更精确的 route 可能永远没有机会执行。单元测试如果只测了 happy path，CI 会显得很安静，线上用户会替系统发现问题。

delete key 的 Boolean logic bug 也一样。键盘事件通常带着多个布尔条件，例如当前是否处于编辑模式、焦点是否在输入框、是否有选中对象、按下的键是否为 Delete。一个条件写反，用户看到的“我按了 Delete，界面没有删掉该删的东西，或者删掉了不该删的东西”。这类 bug 很适合由 Evaluator 通过真实交互抓出来，因为它需要把 UI 状态、键盘输入和最终效果串起来看。

Evaluator 的补位

CI 更擅长检查已经写进测试集合的断言；Evaluator 补的是“产品被真实使用时会不会露馅”。在 00:31:45–00:31:53，Ash 明确说这些错误被抓到，是因为 Evaluator actually using the app。这里的“使用”包括打开页面、点击、按键、观察状态变化和把结果写回 critique。

这里还有一个容易被忽略的点：Evaluator 抓到的错误并不高深，甚至很基础。长程 Agent 的失败常常败在这种基础处，因为长时间运行会制造大量小状态、小前提和小交互。一个按钮、一个 route、一个焦点状态都可能成为后续功能的地基。地基歪了，后面的 sprint 会在错误假设上继续堆东西。

5.2 27 条 Criteria：具体性带来可行动反馈

00:32:08–00:32:24，Ash 把数字说出来：这个示例应用里，Generator 和 Evaluator 最终确定了 27 条 contract criteria。这里需要先定义本章负责的术语。Contract criterion，中文可写作“契约准则”，

指 Contract 中一条原子化、可验证、带边界条件的完成标准。它比“功能要好用”更窄，通常能回答三个问题：检查什么、用什么证据检查、失败后应该改向哪里。

可以把一份 Contract 记成一个准则集合：

$$C = \{c_1, c_2, \dots, c_{27}\}$$

其中：

- C 表示 Generator 和 Evaluator 在写代码前协商出的 Contract。
- c_i 表示第 i 条契约准则，也就是一个可单独检查的完成条件。
- 27 是 Ash 在示例应用中给出的准则数量，它没有通用上限含义，只是这次应用达到可行动反馈所需的粒度。

“可行动反馈”也需要在这里第一次定义。Actionable critique，中文可写作“可行动反馈”或“可执行批评”，指 Evaluator 给出的反馈能把一条准则、一个观察证据和一个修复方向连在一起。低质量反馈会写成“体验差”；可行动反馈会写成“当用户在 play mode 中按 Delete，选中对象没有按准则 c_i 消失；请检查键盘事件分支中的 Boolean 条件”。前者只能制造焦虑，后者能启动修改。

Ash 在 00:32:24–00:32:37 给了因果链：vague criteria 会产生 vague critiques，Generator 只会耸耸肩继续做点什么；granular criteria 会让 Agent 知道“我要修这一行”。这句话很硬，也很工程。标准越像雾，反馈越像雾；标准越像坐标，反馈越像导航。

粒度不足会变成假反馈

“整体再 polished 一点”“更像真实产品”“别出 bug”都属于低分辨率要求。它们表达了人类不满，却没有把不满投影到可检查的状态。长程 Agent 需要的是能落到 UI 事件、API 行为、状态机边界、视觉层级或测试步骤上的准则。

27 条准则的教学意义也在这里：Contract 不要求 Planner 从一开始把产品写死；它让 Generator 和 Evaluator 在进入实现前，把“done-ness”拆成足够多的检查点。一个复杂应用通常同时有交互、视觉、数据、错误处理、性能和边界状态。只有一条“应用可用”，Evaluator 很难指出问题；拆成 27 条，Evaluator 就能说第 9 条通过、第 14 条失败、第 21 条证据不足。

What the evaluator actually caught

Contract criterion	Evaluator finding
Rectangle fill: click-drag fills area	FAIL. Only places tiles at drag start/end. fillRectangle exists but isn't triggered on mouseUp.
Delete key removes placed entities	FAIL. Handler requires both selection AND selectedEntityId. Clicking only sets one. Condition should be OR.
Reorder animation frames via API	FAIL. PUT /frames/reorder defined after /{frame_id}. FastAPI matches 'reorder' as a frame_id int, returns 422.

图 17: Evaluator 抓到的失败项包括拖拽填充、删除键布尔逻辑、FastAPI 路由顺序等，说明真实使用比只看测试结果更接近产品质量。¹⁷

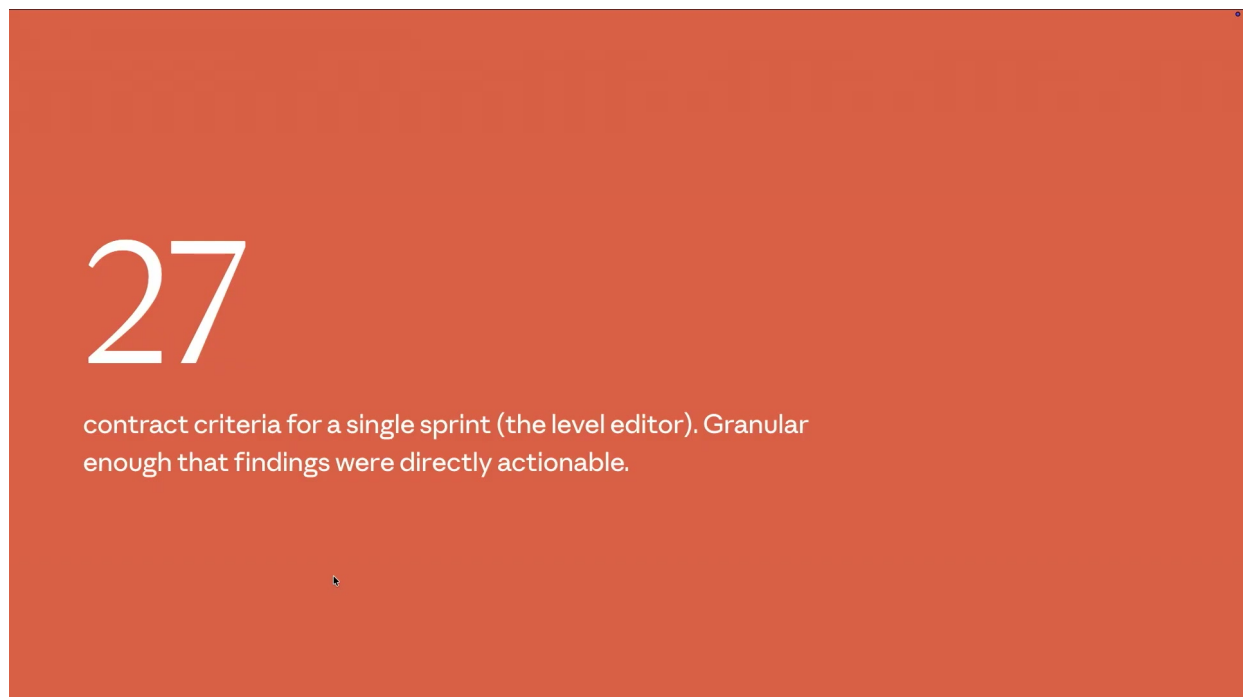


图 18: 一个 level editor sprint 被拆成 27 条 contract criteria；粒度足够细，Evaluator 的发现才可直接转成修复动作。¹⁸

¹⁷ 视频画面时间区间：00:31:28-00:32:40。若为视频帧截图，时间区间对应原视频画面。

¹⁸ 视频画面时间区间：00:31:28-00:32:40。若为视频帧截图，时间区间对应原视频画面。

5.3 读 Trace 像读 Stack Trace

00:33:35–00:33:55, Ash 说构建这个系统的主要 debugging loop 是 reading the traces, 并把它类比为 reading a stack trace。这里的 Trace, 中文可写作“运行轨迹”, 指 Agent 在一次运行中的完整中间输出、工具调用、判断路径、观察证据、失败记录和角色间交接痕迹。它比普通日志更宽, 因为普通日志多半记录程序发生了什么; Trace 还要记录模型为什么在当时的上下文里认为那样做合理。

Stack trace 告诉工程师程序在哪个调用栈上崩了; Agent trace 告诉 harness builder 判断在哪个上下文节点上偏了。前者沿着函数调用找异常, 后者沿着模型的观察、解释、选择和工具调用找偏差。Ash 在 00:33:40–00:33:49 说得很具体: 读 Agent 实际做了什么, 找到它的 judgment 与人类判断 diverged 的位置, 然后 tune prompt。

Trace 与日志的差别

日志像监控录像, 记录门什么时候开了、按钮什么时候被按了。Trace 更像把检查员的笔记也放进来: 它看到什么证据、忽略了什么证据、为什么觉得可以通过、为什么把一个 bug 推迟处理。长程 Agent 的 harness 调试需要后者, 因为主要故障常常出在判断链条, 单个命令退出码给不出足够解释。

这个过程没有捷径。Ash 在 00:33:15–00:33:28 说团队花了大量时间调小 layout bugs、edge cases, 并把这些判断喂回 prompts。换句话说, Prompt tuning 在这里要从 trace 中提取人类和模型的判断差异, 而非凭空写更强硬的命令。例如早期 QA Agent 发现 bug 后说 “fix it later, might take 2 weeks”, 然后就结束了。这暴露的是评价职责边界问题, 工具能力并非主因: Evaluator 需要被要求把当前阻塞写成可执行批评, 不能把产品缺陷外包给未来。

对于读者来说, 读 trace 的正确姿势也更朴素: 先重建当时的上下文。Evaluator 当时看到了哪些文件? 它有没有真正打开页面? 它是否只读了 diff? 它的 rubric 是否包含这个边界? 它有没有把 “我不喜欢” 误写成 “通过”? 只有这些问题回答清楚, prompt 改动才有方向。

The debugging loop

Read the evaluator's logs. Find
where its judgment diverges from
yours. Update the prompt. *Repeat.*

图 19: Trace debugging 的核心循环: 读 evaluator logs, 定位模型判断与人类判断分叉的位置, 更新 prompt 后再跑。

5.4 用另一个 Agent 辅助分析 Transcript

00:33:56–00:34:14, Ash 给了一个很实用的 tooling tip: 把 agent transcripts pipe 到文件里, grep 它们, 让另一个 Agent play through them, 再更新 prompts。这个建议的重点是把运行轨迹变成可检索、可复盘、可比较的工件; 另一个模型只是辅助分析员, 不能替代人类对关键判断的校准。

一个可复刻的流程可以写成四步:

1. 把每次运行的 transcript、工具调用、Evaluator critique、最终产物和失败状态落到文件系统。
2. 用 grep 或结构化查询找高频失败词, 例如 “later”、“should”、“probably”、“not critical”、“unable to verify”。
3. 让另一个 Agent 只做分析员, 沿着 transcript 复盘 “人类会在哪里判失败, 模型在哪里判通过”。
4. 把复盘结果变成 prompt、rubric 或 contract criterion 的改动, 再跑下一轮。

这就是对 harness 本身的闭环。前面的 Generator 和 Evaluator 在改应用; 这里的 trace 分析在改 harness。两层循环不要混在一起。应用循环问 “这个功能是否达标”; harness 循环问 “为什么我们的 Evaluator 没有稳定地产生好判断”。

本章第一性结论

具体性是长程 Agent 的控制带宽。27 条 contract criteria 让 Evaluator 能给出可行动反馈; Trace 让人类能看到模型判断与人类判断分叉的位置; prompt 和 rubric tuning 把这些分叉点写回下一轮运行。

5.5 本章小结

本章把 Ash 在 00:31:28–00:34:14 的短段落展开成一套工程纪律。Evaluator 的价值首先体现在真实使用应用, 抓住 CI 漏掉的 route ordering、delete key Boolean logic 和 production-only 风险。然后, Contract 必须细到准则级; 示例中的 27 条 criteria 说明, 粒度越清楚, critique 越能落到修复动作。最后, Trace 是 harness builder 的主调试材料。读 trace 像读 stack trace: 沿着证据和判断链条找分叉点, 再把人类判断写回 prompt、rubric 和下一版 Contract。

6 Frontier 移动后, Harness 也要删减

00:34:14 之后, Ash 把话题从 “怎样让 Evaluator 更具体” 转到 “模型变强以后, Harness 怎样变轻”。这段很容易被误读成 harness design 失效了。更准确的理解是: Harness 一直是对当前模型失败模式的外部补偿。模型的失败模式变了, 补偿结构也要变。继续保留旧补丁, 会把曾经有用的工程纪律变成今天的摩擦。

6.1 Harness 是对模型短板的外部补偿

本节先定义两个术语。Model-specific spiky behavior, 中文可写作 “模型特异的尖刺行为”, 指某个模型在能力曲线上的不均匀表现: 它可能很会写代码, 却在上下文快满时急着收尾; 可能连续规划两小时, 却在主观审美上过度宽容; 可能能用工具, 却对某类 UI 状态判断迟钝。它像山脊上的尖刺, 局部很高, 旁边突然掉下去。Harness 的作用就是在这些低洼处架桥。

¹⁹视频画面时间区间: 00:33:38–00:34:14。若为视频帧截图, 时间区间对应原视频画面。

Harness simplification, 中文可写作“脚手架简化”或“Harness 简化”, 指当某个外部机制对应的失败模式已经明显减弱时, 把那段机制移除、合并或降低频率。它的目标是让控制结构继续匹配模型能力, 而非追求少写代码的洁癖。一个原本用于防止 context anxiety 的强制 fresh session, 如果在新模型上已经不再带来收益, 就会增加交接损耗、压碎上下文连续性, 并让成本变高。

为什么要删减 Harness

Harness 是补偿工具, 不能当成信仰。每个外部循环、handoff、fresh context、Evaluator cadence 都应该对应一个明确失败模式。读 trace 后发现失败模式消退, 就要考虑简化; 读 trace 后发现新失败模式出现, 就要加新的约束。

Ash 在 00:34:38–00:34:52 给了判断原则: 要摸清每个模型的 spiky behaviors, 然后让 harness 去填模型的 gap。这句话把“模型能力”和“系统工程”接在一起。所谓 frontier 移动, 不只是一张 benchmark 表变好; 它会改变哪些 scaffold 有必要, 哪些 scaffold 会成为负担。

6.2 从 Opus 4.5 到 Opus 4.6: 删掉多余循环

00:34:53–00:35:15, Ash 说他们把 session 之间的 context resetting 整体拿掉了。原因很具体: Opus 4.5 有严重 context anxiety; Opus 4.6 在 post-training 之后, 对长上下文和 compaction 的承受能力更好, 一个 continuous session 加 compaction 就足以处理很长的 session。

这并不表示 compaction 已经完美。Section 6 结尾的五条 takeaway 还会强调, compaction 不能保证 coherence, lossy summaries 会漂移。这里的意思更窄: 在这个 harness、这个任务、这个模型版本上, 强制每个小阶段重开上下文的收益下降了。旧机制解决的是 Opus 4.5 的焦虑收尾问题; Opus 4.6 的主要瓶颈转移后, 同一机制就不再总是划算。

00:35:15–00:35:35, 第二个被调轻的是 sprint decomposition。Ash 说它对 Opus 4.5 很关键, 但 Opus 4.6 能够 coherent 地 hold 一个 2-hour continuous build, 不需要被一项功能一项功能地硬喂。这里可以第一次定义 hill climb, 中文可写作“爬坡式改进”: 在 Evaluator 信号指引下, 系统从当前产物出发逐步修正, 像沿着局部坡度往更高质量位置移动。它的风险是容易卡在局部最优, 例如架构一开始就错了, 后面每轮只是在坏结构上补洞。Opus 4.6 能吃下更长的连续生成段, 意味着系统可以先生成更完整的候选产物, 再让 Evaluator 反馈, 减少过早分段带来的局部爬坡。

00:35:37–00:35:55, Evaluator cadence 也变了。过去每个 sprint 都跑 Evaluator; 现在可以等模型完成一个更大的 one-shot generation 后再把 critique 传回去。核心结构还在: Planner、Generator、Evaluator 仍然是系统骨架, 文件系统 handoff 仍然承担共享状态。变化发生在循环的频率和切片方式上。

删减的三个层次

这段案例里, 删减至少有三层: 第一, 减少 forced fresh sessions; 第二, 减少把任务切成很碎 sprint 的硬约束; 第三, 把 Evaluator 从每个小 sprint 后运行, 调整到更大生成段之后运行。系统骨架保留, 局部节奏变轻。

Ash 在 00:35:56–00:36:08 概括得很准: 原来的 harness 对 4.5 是合适的; frontier 移动后, 团队跑了 simplified version 来验证。这里给读者一个现实提醒: 不要把今天的最佳实践冻结成明天的模板。长程 Agent 的工程方法像自动控制系统, 控制器必须随被控对象变化而重新调参。

Opus 4.5 to 4.6: harness on a diet

Component	On 4.5	On 4.6
Context resets between sessions	Essential (context anxiety)	Dropped entirely
Sprint decomposition	Load-bearing	Removed. Single continuous build.
Evaluator cadence	Per-sprint	Single pass at end
Planner	Required	Still required

图 20: 从 Opus 4.5 到 4.6, 部分 scaffold 被删减: context reset、sprint decomposition 和 per-sprint eval loop 的必要性下降, Planner 仍保留。²⁰

The simplified harness, end to end

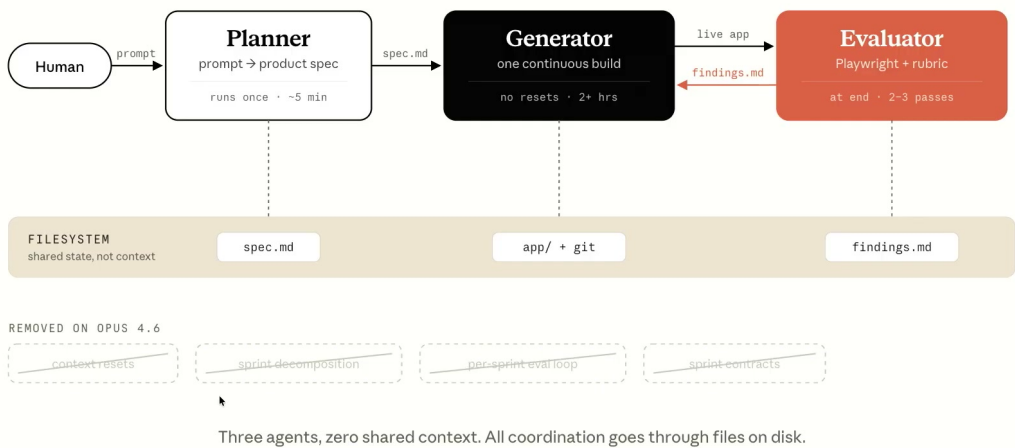


图 21: 简化后的端到端 Harness: 三类 Agent 零共享上下文, 协作通过 spec.md、app/ + git、findings.md 等文件完成。²¹

²⁰ 视频画面时间区间: 00:34:14-00:36:20。若为视频帧截图, 时间区间对应原视频画面。

²¹ 视频画面时间区间: 00:34:14-00:36:20。若为视频帧截图, 时间区间对应原视频画面。

6.3 DAW 示例：能力边界也要写清楚

00:37:04–00:37:56, Ash 展示了一个 DAW 示例。DAW 是 Digital Audio Workstation, 数字音频工作站, 可以理解成用于编曲、录音、排布音轨和播放音乐片段的应用。这个例子很适合讲能力边界, 因为它同时展示了强项和短板: Agent 能设置 tempo、key, 铺 melody, 建立 drum tracks, Evaluator 还能进入应用里测试; 但 Claude 当时不能听, Ash 很直白地说音乐质量 pretty trash。应用本身 fleshed out, 音乐审美很弱。

这正是 Harness 设计要写清楚的边界。对于 DAW, Evaluator 可以检查 UI 是否有 tempo 控件、key 选择、旋律轨、鼓轨、播放按钮、音轨编辑状态; 也可以测试交互是否生效。但如果模型听不到声音, Evaluator 对“这段音乐好不好听”的判断就缺外部证据。此时有三种更诚实的处理方式: 接入能分析音频的工具, 让人类在关键点听一遍, 或者把 Contract 限定为“结构完整、可操作、可导出”, 不把音乐审美伪装成已验证事实。

能力边界要进入 Contract

“应用做得不错”不能自动推出“音乐作品好听”。DAW 示例提醒读者: 当模型缺少某种感知通道, 例如听觉、真实设备反馈、生产流量或领域专家判断, Contract 必须显式写出可验证范围和不可验证范围。

这个例子也解释了为什么 Ash 说这是 Andrew 前面 meter curve 的现实体现。一个模型版本之前难以跑通的任务, 现在经过简化 harness 和少数几轮反馈能做出完整应用。能力提升并不意味着所有目标都可由模型自评完成; 它意味着系统可以把更多精力放在剩余缺口上, 例如音频质量评估。

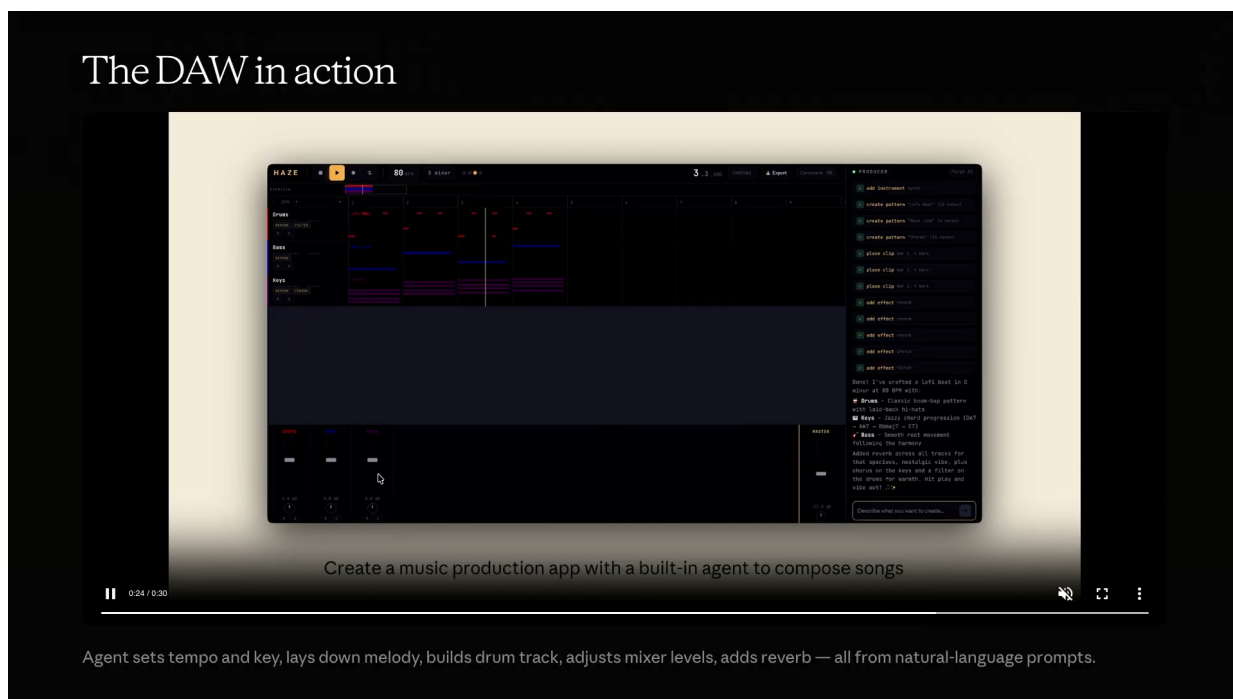


图 22: DAW demo 展示 Agent 设定 tempo、key、旋律、鼓组与 mixer 的过程; 即使音质判断受限, 界面行为仍可由 Evaluator 检查。²²

²² 视频画面时间区间: 00:37:04–00:37:50。若为视频帧截图, 时间区间对应原视频画面。

6.4 普通开发者可以复刻的原语

00:37:58–00:38:54, Ash 明确说听众不需要 Anthropic 的 internal harness 才能开始做类似事情。可复刻的原语已经在产品和工具生态里逐步露出：Claude Code 的 auto mode、custom sub-agents、一个更 harsh 的 QA system prompt、详细 rubric、Web 应用里的 Playwright MCP 或 Claude for Chrome MCP、native app 场景中的 computer use，以及用 skills 打包 grading rubrics。

对普通开发者来说，可以把这些原语装成一个最小可行 harness：

1. 先把 Builder 和 QA 分成不同 sub-agent，至少给 QA 一个独立 system prompt。
2. 给 QA 写 harsh rubric，明确它要找失败证据，不能为了礼貌提前放行。
3. Web app 用 Playwright MCP 或 Claude for Chrome MCP，让 Evaluator 打开真实页面并执行操作。
4. Native app 用 computer use 或等价工具，让 Evaluator 看到屏幕状态和交互结果。
5. 把 rubric、检查清单和项目约束打包进 skill，让它们进入日常开发流，避免散落在一次性 prompt 里。
6. 把 transcript 和 trace 落盘，定期读它们，再决定保留、合并或简化哪段 scaffold。

这里的关键是先从一个垂直场景开始。比如只做“前端小游戏生成”，rubric 可以先覆盖可玩性、键盘控制、碰撞、失败/胜利状态、视觉层级和截图证据。不要一开始就写一个声称能评估所有软件质量的通用 QA Agent。通用性来自多次 trace 复盘后的收敛，第一版 prompt 的雄心给不出这种收敛。

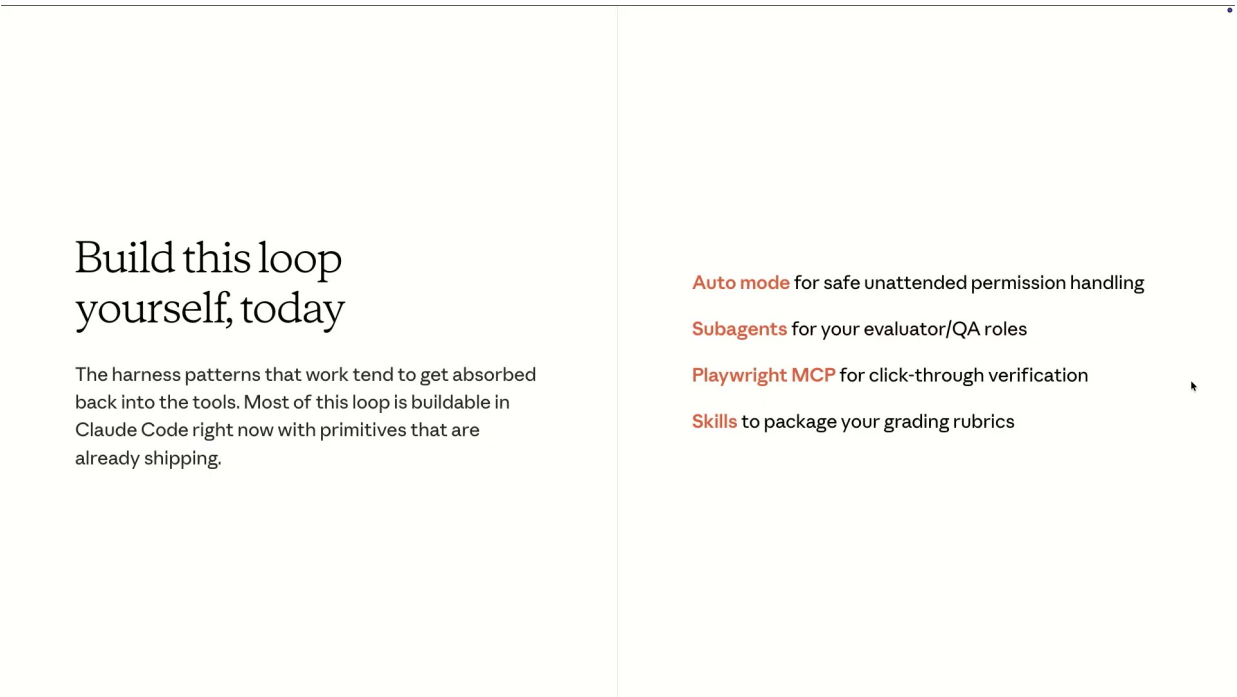


图 23：普通开发者可复刻的 Harness 原语：auto mode 处理权限，subagents 承担 evaluator/QA，Playwright MCP 做点击验证，skills 封装 rubric。²³

²³ 视频画面时间区间：00:38:15–00:39:40。若为视频帧截图，时间区间对应原视频画面。

Takeaways

- 1 Self-evaluation is a trap. Use an adversarial evaluator.
- 2 Compaction doesn't cure coherence drift. Structured handoffs do.
- 3 Make subjective quality gradable with rubrics the model can apply.
- 4 Read the traces. They're your primary debugging loop.
- 5 Delete scaffolding when the model catches up. The frontier moves.

图 24: 五条 takeaway 压缩了整段 workshop: 对抗式 evaluator、结构化 handoff、rubric、trace reading, 以及随 frontier 移动删减 scaffold。²⁴

6.5 本章小结

Ash 在 00:38:54–00:39:53 给了五条 takeaway, 本章把它们放回 harness evolution 的主线中:

1. 自评是陷阱。Generator 对自己的产物有天然宽容, 长程运行会放大这个偏差; 用 adversarial evaluator 把评价压力外置。
2. Compaction 不能等同于 coherence。压缩摘要会丢信息, 丢失的边界条件会在后续运行里漂移。
3. Structured handoffs 和 clean contexts 很有用。文件系统、明确 Contract、角色隔离和可读工件能降低长程混乱。
4. Subjective quality 可以被评分。前提是**把强观点写成 rubric**, 用参考例和人类判断校准它。
5. 先陪模型坐一会儿, 读 trace, 再决定 scaffold 的去留。Frontier 移动后, 只有 trace 能告诉团队哪些旧补偿已经过期, 哪些新尖刺正在出现。

本章的硬结论是: Harness 要随模型一起演化。Opus 4.5 需要更多 fresh context、sprint decomposition 和高频 Evaluator; Opus 4.6 可以承载 2 小时 continuous build, 于是一些外部循环可以简化。DAW 示例说明, 能力增强和能力边界会同时出现。真正可靠的长程 Agent 工程, 不会把 scaffold 越堆越厚; 它会根据模型特异的尖刺行为, 把该保留的证据链保留, 把过期的补偿拿掉。

7 Q&A 上半场: 复用、信任、重跑与模型选择

00:40:05 之后进入 Q&A, 上半场把前面讲过的 Planner、Generator、Evaluator、Contract、Rubric 和 Trace 放回真实工程选择里: 哪些规则可以复用, 哪些规则要项目化; 什么时候让 Agent 继续爬坡,

²⁴视频画面时间区间: 00:38:15–00:39:40。若为视频帧截图, 时间区间对应原视频画面。

什么时候让它重来；什么模型该承担什么角色；一个生成出来的应用如何留下足够证据，供人类或下一轮 Claude Code 接手。

7.1 哪些 Rubric 能复用，哪些必须项目化

第一个问题来自观众 Joan：团队通过读日志改进 Evaluator，这种改进是逐项目调参，还是可以跨项目复用的“秘方”（00:40:18–00:40:36）？Ash 的回答给出一条很实用的边界：优先复用“模型共同弱点”上的规则，把“项目口味”留给局部定制（00:40:36–00:41:20）。

决策规则：先复用模型弱点，再定制项目差异。如果某条 Rubric 针对的是模型普遍容易犯的错，例如前端设计里常见的模板化审美、层次混乱、按钮像完成了但行为没有接上，那它适合沉淀成通用 Evaluator 规则。如果某条规则只描述某个产品的特殊业务、品牌语气、行业规范或团队偏好，它可以保留在该项目的 Harness 中，但不要伪装成通用能力。

这条规则对应一个很硬的判断：如果调参只是在生成某一种非常具体的应用，Ash 认为它已经接近“自己提示 Claude Code 去做同一个东西”（00:40:41–00:40:53）。真正值得沉淀的部分来自跨项目反复出现的模型弱点。例如他们知道什么算“好设计”，也知道什么算 AI slop，也就是低密度、模板化、像模型默认审美一样堆出来的界面；这些正反样例可以迁移到其他 Web App，甚至迁移到非 Web 的产品质量判断上（00:40:59–00:41:20）。

这里读者容易少看一层：Rubric 的复用对象瞄准“判断误差”，业务需求只提供局部约束。业务需求像地图上的目的地，每个项目都不同；判断误差像驾驶员的视觉盲区，同一个驾驶员换城市后仍然可能看漏后视镜。Harness 复用要抓后者。

7.2 新上下文循环与单会话压缩如何取舍

这一段 Q&A 还补上了 Ralph Loop 和 compaction 的选择问题。听众先把 Ralph Loop 解释成一种 smart zone 管理：早期经验里，长上下文并非每个位置都同样可靠，任务会被切到模型仍较清醒的区间内；他还提到从旧的约 40% 经验，到 1M context 下可能约 100K token 的 smart zone（视频 00:41:38–00:42:08）。这个提问很有用，因为它把“切片”从形式上的多 session，拉回到可靠性边界：让模型停留在判断质量较高的区域。

前文讲过，Ralph Loop 倾向于把每轮任务放进新上下文：上一轮通过文件、提交、计划或摘要交接，下一轮用干净窗口继续。这种做法粗糙，但它能把上下文腐化的风险压低，适合任务边界清楚、每步都能被文件状态承接的工作。Compaction 则把长会话压缩成摘要继续跑，优点是对话连续，缺点是压缩会丢细节，尤其容易丢掉“为什么这么做”和“哪些方案已经失败”这种反事实信息。

决策规则：按证据形态选择状态载体。如果任务可以被文件、测试、git 历史和清晰 Contract 接住，Ralph Loop 式的新上下文更稳，因为每一轮都从较干净的状态出发。如果任务需要持续保留模糊意图、设计偏好、长链讨论和多轮审美判断，单会话加 compaction 可能更顺手，但必须用 Trace 与外部工件弥补摘要损耗。最后以 eval 做裁决：比较同一任务在新上下文循环和单会话压缩下的完成率、返工率、误判率和人工接手成本。

这条规则也解释了为什么 Andrew 会说模型变强后 Harness 需要调整。Opus 4.6 能更稳定地承受较长连续构建时，一些强制新 session 的结构可以被拿掉；但这不等于 compaction 天然等于连贯性。压缩只是把更多历史带进下一轮，连贯性还要靠 Contract、真实测试、Evaluator 和 Trace 共同维持。

7.3 看着浏览器跑：信任缺口与自动化验证

接着有观众问 Playwright MCP 是否就是推荐路径，是否应该打开浏览器看模型在里面操作、必要时人工接管（00:43:53–00:44:26）。Ash 的建议很直接：用 Playwright MCP，或者更偏浏览器控制的 Claude for Chrome MCP（00:44:24–00:44:34）。但他随即把问题从“看不看浏览器”推进到“为什么想看浏览器”。

信任缺口（trust gap）指人类尚未相信 Agent 能独立完成、验证并报告任务时产生的监控冲动。表现上是想盯着浏览器、看每次点击、随时准备接管；机制上是缺少可检验的运行证据。弥补它要靠 Agent 留下足够可信的自动化证据：Trace、浏览器动作、网络错误、控制台错误、截图、可复现测试结果，以及 Evaluator 的具体判据。

Ash 说，想看它操作，今天很大程度上是 trust gap：理想工作流是把任务放出去，相信它会执行、测试、给出正确证据，然后人回来验收（00:44:34–00:44:51）。早期当然要观察，要读 Trace，要看浏览器行为，直到 Harness 通过多轮失败分析建立信心（00:44:50–00:44:57）。这像调试一条自动化生产线：初期工程师站在旁边听每个轴承的声音，成熟之后看仪表盘和质检记录。

浏览器自动化的最低可信证据也被说得很具体。Ash 提到，在 Full-stack 应用上，Opus 4.x 已经可以较可靠地读取网络错误和控制台错误，实际导航应用，在需要时缩放页面；模型视觉能力也足以识别元素上的文字重叠等 UI 问题（00:45:03–00:45:34）。因此，浏览器承担 Evaluator 获取外部证据的传感器角色，装饰性演示只是副产品。没有网络错误、控制台错误、视觉检查和交互路径，所谓“我已经测试过了”就只是自我报告。

7.4 重跑、丢弃与继续迭代

第三类问题问得很像真实产品场景：如果用户让 Harness 生成一个“很酷的游戏”，结果不满意，能否重启并说“make it better”？Evaluator 会继续推动改进，还是某个时刻放行一个平庸结果（00:45:45–00:46:45）？

Ash 先承认一种工程接口：如果需要人类介入，可以在循环中加 hook，让流程在某个点暂停并接受人类输入（00:46:45–00:46:56）。但他们真正意外的是另一件事：在 4.6 系列模型，尤其 Sonnet 4.6 和 Opus 4.6 上，Generator–Evaluator 模式很愿意把已经做了十轮的东西整体丢弃，从头开始（00:46:56–00:47:18）。触发条件是它无法有效顺着 Evaluator 的 Rubric 继续 hill climb，也就是无法沿着评价信号逐步变好（00:47:18–00:47:24）。

不要把“继续迭代”当成默认美德。当 Evaluator 的反馈反复指向同一类结构性失败，例如交互模型错了、视觉层级无救、技术路线导致每个补丁都制造新问题，继续补丁会把坏结构腌得更深。此时更便宜的动作是保留需求、Trace 和教训，换一个干净起点重做。

这解释了前文为什么要把 Builder 和 Evaluator 分离。Ash 观察到，单独的生成器容易“为自己的作品骄傲”，抗拒整体重启；独立 Evaluator 更可能明确说：这条路明显走不通，请丢弃当前方案并重来（00:47:45–00:48:19）。他还把这与人类写代码类比：工程师也常在上下文变脏、代码结构混乱时开一个新起点（00:48:19–00:48:37）。

Andrew 补上一条接续规则：生成出的代码库可以打开到 Claude Code 里继续开发（00:48:35–00:48:44）。对于复杂游戏应用这种任务，运行时间可能是 3 小时，也可能是 20 小时，因此中间形态可能需要更细的反馈循环（00:48:44–00:49:04）。决策上可以这样分：

- 如果失败集中在局部 bug，继续让 Generator 根据 Contract 和 Trace 修补。

- 如果失败集中在产品结构、交互范式或技术路线，允许 Evaluator 宣告“本轮不可爬坡”，保留证据后重跑。
- 如果人类偏好本身还没有被写进 Rubric，通过 hook 插入一次人类反馈，再把反馈固化到 Harness，避免人类长期补锅。

7.5 Planner 何时回环，何时保持高层

有观众把系统类比成 PM、工程师和 Evaluator，并问 Planner 是否需要回到循环中，防止工程师在沙盒里玩太久、范围蔓延（00:49:06–00:49:37）。Ash 的答案延续了第 4 节的原则：Planner 提供原始规格和产品外轮廓，Builder 与 Evaluator 负责把它压成具体特性、测试和 Contract。

他们会把 PM 生成的主规格定期插入 Builder 和 Evaluator 的会话里，让两者始终知道“我们到底还在建什么”（00:49:44–00:50:10）。Builder 与 Evaluator 的主要职能，是找出满足这个规格的精确功能集、测试和契约（00:50:10–00:50:23）。Planner 的工作则是画出产品的“硬外线”：大致边界、方向、不可越过的目标范围（00:50:25–00:50:41）。

决策规则：Planner 负责外轮廓，Contract 负责贴地。当问题是“目标是否还对齐原始规格”，把主规格重新注入 Builder-Evaluator 会话；当问题是“某个功能是否可测、是否满足 done-ness”，让 Builder 和 Evaluator 继续谈 Contract。只有当原始规格本身已经错了、过时了或商业目标改变了，Planner 才需要重新上场。

这条规则也防止 Planner 变成另一个频繁打断的管理者。Ash 明确说，他们不希望 Planner 成为核心循环的一部分；它不应该反复介入并说“这个功能不可能，别做了”（00:50:25–00:50:53）。如果需要更复杂的任务分解，可以把 Generator-Evaluator 对放入多步骤 workflow。例如生成评测时，先有合成数据集的生成器和 QA Agent，再交给集成者及其 QA Agent，最后再有收尾检查（00:51:12–00:51:45）。这里改变的是 workflow 拓扑，核心压力仍然来自每个 Builder 旁边的 Evaluator。

7.6 按角色选模型，而非按品牌选模型

随后观众问：团队如何比较不同模型，例如 Opus 4.6、Opus 4.5、Sonnet；Harness 与模型是否共同演化到难以直接横向比较（00:52:01–00:52:32）？Andrew 的回答把模型选择从“谁最强”改成“谁适合这个角色、这段提示、这类任务”。

模型-角色匹配（model-role matching）指把模型当作 Harness 中的角色资源来分配，避免按单一排行榜选择。Planner 可能需要更强的长程规划和方向感，Generator 可能需要高性价比执行，Evaluator 可能需要视觉、工具使用和挑错稳定性。真正要评估的是三元组：模型、提示、角色任务。

Andrew 回顾了两代 Harness 的差异。早期长程 Agent 的 initializer 会生成 200 个左右的功能点，后续循环被迫逐条执行，这会把错误设计决定锁死在后续任务里（00:52:32–00:53:05）。到 Opus 4.6，他们更能给出较通用的创意方向，再让 Generator-Evaluator 循环发展具体实现（00:53:05–00:53:20）。所以模型选择会深刻影响 Harness 设计（00:53:15–00:53:22）。

成本约束也直接进入角色分配。Andrew 说，如果没有成本问题，当然可以把所有东西都交给 Opus 4.6；现实中常见模式是 Opus 4.6 做 planning，Sonnet 4.6 做 coding 或 execution（00:53:22–00:53:37）。但这不能停在经验口号上：如果为不同角色构建特定 sub-agent，就要为“某个模型 + 某个 prompt + 某个任务”做 eval，观察表现后再优化（00:53:37–00:53:50）。

一个紧凑的形式化写法是：

$$R^* = \arg \max_{m,p} Q(r, m, p) - \lambda \cdot Cost(m)$$

- R^* ：某个角色的实际选择，例如 Planner、Generator 或 Evaluator。
- r ：角色任务，包含规划、编码、浏览器验证、视觉挑错等。
- m ：候选模型，例如 Opus 4.6 或 Sonnet 4.6。
- p ：该角色的提示、Rubric、工具权限和上下文注入方式。
- $Q(r, m, p)$ ：在该角色评测上的质量。
- $Cost(m)$ ：调用成本、延迟和可用性等工程成本。
- λ ：团队对成本敏感度的权重。

公式的意思很朴素：模型选择是在给角色配工种，品牌投票这个类比会误导判断。强规划模型放在外轮廓上，高性价比比执行模型放在重复编码上，擅长浏览器证据和视觉检查的模型放在 Evaluator 上；每个组合都要通过 eval 说话。

长寿命产品工件：让下一轮能接手。 最后一个属于本节时间窗的问题，是从一次性应用走向长寿命产品：如果几天或几周后还要修改，应该留下什么 artifact，让未来实例知道此前发生了什么、哪些地方能改、哪些地方应该改（00:53:52–00:54:08）？

长寿命产品工件（long-lived product artifact）指为了让生成应用在未来仍可维护而持久化下来的证据与导航材料。代码本身只是其中一部分；更关键的是围绕代码留下接续线索：状态 JSON、带时间戳的尝试记录、评估结果、已修复缺陷、文件结构说明、关键截图、必要时的屏幕录制，以及能让人类或 Claude Code 快速恢复任务上下文的高层文档。

Ash 说他们仍在探索这件事。当下常见工作流是：把任务放到远端服务器上跑，讲座结束后回来检查，再手动在代码里打磨粗糙边缘（00:54:08–00:54:36）。但 Harness 的底层状态选择很清楚：他们默认用文件系统保存状态，因为另一个模型可以很容易地 grep、浏览并接上前面的线索（00:54:36–00:54:55）。

最小可用的接续包有两类。第一类是 JSON 状态文件，里面不断写入 learning 和 state，作为给下一个模型的 breadcrumbs（00:54:55–00:55:17）。这个文件的形状可以很具体：尝试了什么，Evaluator 发现什么 bug，实现了什么 fix，这个 fix 是否有效，最后状态是什么，并附带时间戳日志（00:55:17–00:55:49）。第二类是实时更新的高层文档，说明文件结构；Ash 在 00:56:00 前已经把这两类文件作为 Claude Code 与人类接续工作的入口。

截图和屏幕录制的价值来自同一逻辑：它们给未来接手者一个产品行为的外部证据。前文 Evaluator 已经会导航应用、截图、读取控制台和网络错误；当产品要跨天维护时，把关键 UI 状态、失败路径、成功路径和复杂交互录下来，可以避免下一轮模型只凭文件名猜测产品意图。对于纯后端任务，结构文档和测试日志可能更重要；对于 Web App、游戏、Dashboard、设计工具，视觉证据就是长期记忆的一部分。

7.7 本章小结

Q&A 上半场把长程 Agent 的设计从“架构图”落回“取舍”。可复用的是模型共同弱点上的 Rubric，项目特有的业务口味要局部化；Ralph Loop 的新上下文与单会话压缩都只是工具，选择依据

是用例和 eval; 看浏览器运行反映的是 trust gap, 成熟 Harness 要用 Trace、Playwright MCP、Claude for Chrome MCP、网络错误、控制台错误和视觉证据来建立信任; 继续迭代并不总划算, Evaluator 应该能在无法 hill climb 时要求重跑; Planner 画外轮廓, Builder 与 Evaluator 用 Contract 把规格落到测试; 模型选择要按角色、提示和成本匹配; 长寿命产品需要 JSON 状态、时间戳日志、文件结构说明、截图或屏幕录制这类可接续工件。说白了, 长程 Agent 的可靠性来自可复用判断、可验证行动和可传递状态三件事。

8 Q&A 下半场: Agent Teams、上下文隔离、Brownfield 与人类介入

8.1 Agent Teams 与专用 Harness 的边界

00:56:21–00:57:10, 一位听众把问题问得很准: 如果 Agent Team 已经允许多个 Agent 相互协作, 主 Agent 也可以审查子 Agent, 那么为什么还要显式写一个 generator-critic Harness? 关键落在控制面。Andrew 先回答说, Claude Code 和 Agent SDK 共享一套底层 Harness, 因此理论上可以把 generator-evaluator 模式做进 Claude Code; Agent Teams 也是一种有用框架, 可以让生成器和评估器通信, 或者让生成器作为主 Agent、评估器作为团队成员。限制在于运行环境和产品目标: Claude Code 通常跑在用户机器上, 长时间运行还会受本机环境影响; Agent SDK 可以放在云端或沙箱环境里长时间运行, 适合作为独立应用部署前的工程底座 (00:57:15–00:58:28)。

Ash 随后补了一层产品判断: Anthropic 对某一时刻的最佳形态没有强行定论, Agent Teams 是内部许多人喜欢后才发布给用户验证的能力, 也可能随着反馈继续调整。他把 generator-evaluator 看成 Agent Teams 思路下的一种子模式: 前端、后端、集成等子 Agent 都可以配自己的 critic, 像一个小工程团队里每个专业角色都有对应 reviewer (00:58:45–00:59:43)。这给读者的决策规则很直接: 如果目标是复刻演讲里的长时间“一次性生成完整应用”实验, 专用 Harness 给出更强的角色隔离、调度、沙箱、日志和重跑控制; 如果目标是在 Claude Code 内部探索多 Agent 协作, Agent Teams 或普通 sub-agent 可能已经够用。

边界规则: Agent Team 是多 Agent 协作的组织形式, generator-critic Harness 是一种更窄、更可控的质量压力结构。前者强调谁能和谁协作, 后者强调谁负责生成、谁负责独立判断、证据怎样交接、失败后怎样重跑。当前工程实践里, 专用 Harness 的价值在于把这些控制点写死; 未来 Agent Teams 可能吸收更多这类模式。

这段问答还给出一个现实边界: 大多数 Claude Code 用户今天的目标通常不在六小时 one-shot 生成一个完整应用, 所以这类长跑 Harness 没有作为默认 primitive 直接塞进产品主路径 (00:59:45–01:00:01)。如果一个团队要采用它, 第一步应当是把目标场景写清楚: 长时间生成新应用、给现有代码库修 bug、跑监控后开 issue、还是让多个子 Agent 并行查资料。不同目标需要不同的 Harness 粒度。

8.2 Evaluator 看输出, 不共享错误思路

01:00:05–01:00:25, 另一位听众追问: critic 是否应该拿到 generator 的完整上下文, 尤其是执行轨迹和中间想法? 讲者的回答很硬: 他们试过, 但会让两个模型流之间的想法变“浑”。这里本章需要给出一个首定义: 上下文污染 (context contamination) 指评估器在接收生成器完整推理、借口、错误假设或自我解释后, 把同一套误判带入自己的评估过程, 导致原本应当独立的检查者也沿着错误地

基继续判断。它像两名代码 reviewer 共用一份有偏的事故报告：第二个人看似独立，实际已经被第一人的叙事带偏。

因此，评估器通常应看产物、Rubric、契约、运行证据和必要 handoff 信息，而不直接继承生成器的完整思路。讲者给出的更有效反馈形式很短：让 Evaluator 指出“这里有问题”，再让 Generator 回到自己的工作里定位原因和修复路径。若 Evaluator 直接说“你在构建时做了 X，所以导致 Y”，它可能把生成器先前的错误解释也包装成诊断结论。讲者特别强调，模型很容易说服自己某件事能工作，或者某件事已经失败；这种自我误判一旦喂给 Evaluator，评估器也会被污染（01:00:25–01:01:06）。

可以把这个输入边界写成一个小公式。评估器的输入应更接近：

$$E = f(A, H, R, V)$$

其中：

- E 表示 Evaluator 的判断结果，例如问题列表、是否通过、是否重跑。
- A 表示 artifact，也就是生成出的应用、代码、截图、日志或可运行产物。
- H 表示 handoff，只包含必要状态、约束、已知测试结果和待确认事项。
- R 表示 Rubric 或 contract criteria，即评估标准。
- V 表示外部验证证据，例如 Playwright 操作、测试输出、浏览器观察和文件检查。

危险的写法是把生成器完整思路 G_{thought} 也塞进去，使 $E = f(A, H, R, V, G_{\text{thought}})$ 。这会让 Evaluator 的判断从“看证据”滑向“继承叙事”。长程 Agent 里，这种污染会累积：一个早期错误假设如果同时进入生成器和评估器，后面几小时都可能围着同一处错地基装修。

听众随后提出一个训练想法：是否可以训练 generator 预测 critic 会说什么，从而更诚实地汇报自己做了什么（01:01:04–01:01:23）。讲者只给了非常谨慎的回应：“maybe we'll work on that”。所以本节只能把它写成 speculative training idea：它是值得研究的方向，尚不能当成已发布机制。今天可落地的做法仍然是隔离评估上下文、控制 handoff 内容、让证据压过解释。

8.3 Rubric 如何更科学

01:02:51–01:03:21，一位听众问到质量衡量：这个 Harness agent pair 的好坏是否只是 vibe check？在 greenfield 应用里做一个新 app 看起来像艺术判断；到了 brownfield 代码库，能否更科学？讲者没有承诺一个跨产品通用分数，他把“科学”落到 Rubric 的细粒度上：前文提过的设计、原创性、工艺、功能四个高层标准，只是设计品味 Rubric 的表层。真正运行时，他们会在 generator 和 evaluator 层面把 Rubric 写得很细，覆盖设计元素、API 设计、代码质量等不同对象（01:03:19–01:03:54）。

这里的“更科学”不等于把主观判断伪装成绝对数学分数。更可靠的做法是把判断对象拆成可观察维度，让 Evaluator 推动 Builder 沿着这些维度爬坡：这个产品一开始在某些 criteria 上是什么状态，经过若干轮 contract、测试和修复后到了什么状态。讲者说，这个信号不太适合跨产品、跨 run 横向比较，但在同一个产品或同一次 run 内很有用（01:03:55–01:05:04）。换句话说，它更像实验室里的同组前后测量，而非行业排行榜。

Rubric 的科学性来自三个动作：第一，把“好看”“好用”“代码质量高”拆成可检查的 criteria；第二，让 generator 和 evaluator 在编码前把测试与 contract 写清楚；第三，保留 run-level 信号，观察同一产物是否沿着这些 criteria 持续改善。它不能消除人的品味输入，却能把品味变成可复查、可迁移、可调参的工程材料。

这也解释了为什么 trace reading 仍然重要。01:01:25–01:02:49 的 traceability 问答里，讲者说他们会让 Claude 带着自定义 prompt 读一堆 traces，先找出 loop 在哪里偏航；但内部最有效的方法仍然是人手读 trace。原因很朴素：只有读过完整轨迹，人才知道模型当时看到了什么、误解了什么、为什么做出那个选择。自动分析 trace 可以做第一遍筛查，人工读 trace 才能校准 Rubric 和 prompt 的真实错位。

8.4 Greenfield 到 Brownfield 的迁移成本

本章还负责首次定义 greenfield 和 brownfield。Greenfield（绿地项目 / 从零项目）指从空白开始构建的新应用，既有代码、历史接口、视觉规范、部署约束和组织惯例较少；它像在空中搭临时展馆，Harness 可以规定 React、Postgres、Node、设计风格、测试方式和目录结构。Brownfield（存量项目 / 既有项目）指已有技术栈、代码、设计规范、历史数据、接口兼容性和团队流程的项目；它像在一栋仍在营业的大楼里改造电路，任何新结构都要避开承重墙、消防规范、旧管线和正在使用的房间。

讲者把这个边界说得很清楚：这套模式对 greenfield 很强，因为它相当 opinionated。演讲里的 Harness 可能默认 React、Postgres、Node 后端，以及一套他们认为好的设计模式；听众的 brownfield app 可能完全使用另一套栈，甚至“好设计”的 Rubric 也完全不同（01:05:03–01:05:31）。所以迁移成本主要有三项：技术栈成本，Evaluator 必须懂现有框架和架构边界；质量标准成本，Rubric 要匹配团队已有 API 习惯、视觉系统、测试策略和代码审查口味；运行控制成本，Harness 必须更谨慎地处理分支、权限、回滚、PR 和审查。

01:10:46–01:12:08 的追问把这个边界推到真实使用：是否已经用它做过非 greenfield 或生产功能？回答依然克制。讲者说，这个具体模式大多延伸到 greenfield；brownfield 需要更多控制，需要项目内测试，需要为项目定制 Rubric 和 pattern。Andrew 同时指出，存量软件开发生命周期里仍有很多可自动化点：自主监控发现问题，生成 issue 或 feature request，再交给 Agent 开 PR，之后进入 pull review，人类在合并前审查。也就是说，brownfield 不应直接照搬“六小时生成新应用”的长跑模式，它更适合拆成监控、开单、修复、PR、review 这些较窄链路。

Brownfield 的风险不在于 Agent 写不出代码，而在于它可能写出“局部看似合理、整体破坏约束”的代码。既有项目的隐性规则通常藏在测试、旧 PR、目录惯例、部署脚本、设计系统、权限边界和团队审查习惯里。Harness 迁移到存量项目时，第一件事应是让 Evaluator 读懂这些约束，再让 Generator 动手。

现实使用边界也在这里出现。有人追问 Anthropic 是否有内部工具或真实 greenfield app 可分享，讲者直说不能谈太多内部 tooling；但很多 Claude Code 的新功能和日常用法吸收了这套经验，例如提示主模型派 sub-agent、监控与 bug fixing loop、生成 effects 时让 separate evaluator 和 generator 处理同一目标（01:12:14–01:13:12）。因此读者能带走的是原则和局部 pattern，不能把演讲理解成 Anthropic 已公开完整内部生产 Harness。

8.5 团队级 Harness、可观测性与版本控制

01:05:57–01:07:27 的问答把话题从单个 Harness 推到组织维护层面：这类 Harness 是否会成为团队拥有的协作工具，甚至成为 Claude Code 的产品能力？Ash 的回答很现实。Anthropic 内部的常见路径是自下而上采用：某个原始想法的提出者需要把 Harness 维护到可组合、可泛化，供不同团队按代码库片区或任务类型改造。他还承认，超长程 Agent 的 observability 仍没有完全解决。换句话说，长程 Harness 的难点也在运维侧：谁维护它、谁看它跑偏、谁把失败模式抽象成通用规则。

Andrew 补充的落地规则回到软件工程基本功：用 version control、commit、PR 管理 Agent 的产物；个人并行探索时用 Git worktree，避免多个功能分支互相覆盖文件系统。这个建议很朴素，但对 brownfield 尤其重要。长程 Agent 一次跑几小时后，若没有提交边界、分支边界和可审查 PR，团队拿到的就可能是一大团无法追责的改动；有了 worktree 和 commit 粒度，Evaluator、人类 reviewer 和后续 Agent 才能按功能片区检查。

8.6 Human-in-the-loop 的最小插入点

Human-in-the-loop（人在回路 / 人类介入）在本章中指：在关键 stop condition、评估节点、权限节点或方向分歧处暂停 Agent，让人输入判断，再让系统恢复运行。它强调少量关键判断点，避免全程人工陪跑，也避免每个 sprint 都开一次会。01:07:42–01:08:22，自称 Jose、来自 Mercedes-Benz Research and Development 的听众把这个问题类比作 Scrum feature team：长跑 Agent 做了两小时后，是否应该有 sprint review，让人看一眼“过去两小时构建了什么”？

讲者的目标很明确：他们在探索怎样构建大多数时候无需人类介入的 Harness。今天当然没有把这种模式用于所有事情；如果确实需要人类节点，hooks 是主要 primitive。一个简单实现是让 Evaluator 触发某种 stop condition，把控制权交回人类，允许人写一条 developer message 或方向修正，然后继续 loop（01:08:29–01:09:16）。这给出一个实用规则：人类介入应当绑定具体停机条件，例如“Evaluator 判定方向不确定”“合并前需要业务 owner 批准”“安全风险超过阈值”“连续两轮无法修复同一失败”；按时间机械插入 review 会把 Harness 重新拖回会议节奏。

听众补充说，若几小时后能 review 一次，也许可以让八小时后的结果更接近自己想要的项目（01:09:33–01:09:52）。讲者没有否认这个价值，但他把问题转成 Harness 设计：这个人工节点应成为永久结构，还是通过 prompt 和 Harness 调参逐渐内化？他们的做法是跑多次生成，例如 10 个不同结果里 3 个成功、7 个随机失败，然后人坐下来读失败样本，调整主 Harness 的 prompt，再继续尝试，直到足够放心让它自主运行（01:09:52–01:10:43）。这比用人类临时补稳定性更接近他们的目标：把失败经验烤进 Harness。

最后一个 trace 问题把人类介入的核心技能讲透。01:13:25–01:15:13，听众问 trace 是否只是 raw output，还是需要让 Agent 特意写某些文件。讲者回答很直白：要读完整 trace。构建 Agent 的重要技能之一，是尽量站到模型的位置理解它当时看到的世界。他用 Claude for Chrome 的例子说明：团队会假想自己闭着眼睛导航网页，每 10 秒睁眼看一张静态页面，再继续点击；这种“把自己放进模型视角”的练习帮助他们理解浏览器使用 Harness 为什么会失败、怎样改 prompt、怎样把经验写进 prompt templates、Claude.md 或 skill（01:13:35–01:15:13）。

Andrew 还顺手提到 Claude Code 已经有 session auto memory，会在运行中持续记下一些小东西（01:14:59–01:15:07）。这能降低重复注入上下文的压力，但它只是一层辅助记忆，不能替代读完整 trace 后把失败模式写回 prompt、Claude.md 或 skill。自动记住碎片像把便利贴贴到桌边，trace 复盘则是在事故后重画流程图，两者服务的工程目的不同。

最小人类介入点有三类：方向性介入，人在目标偏离时给新约束；权限性介入，人在合并、发布、访问敏感系统前批准；学习性介入，人在失败 run 后读完整 trace，把误判模式写回 prompt、Rubric、Claude.md 或 skill。前两类保护现实系统，第三类让 Harness 下次少犯同类错误。

8.7 本章小结

本章把 Q&A 下半场整理成六条边界条件。第一，Agent Teams 能表达 generator-evaluator 结构，但专用 Harness 目前给长时间 one-shot 生成更多控制点：角色、沙箱、日志、重跑、证据和失败处理。

第二，Evaluator 应主要看产物、Rubric、contract、handoff 和外部验证证据；让它继承 Generator 的完整思路会带来上下文污染，使同一套错误假设同时支配生成与评估。第三，把 Rubric 写科学，靠细粒度 criteria、同一 run 内的前后测量、contract 与 trace 校准，不能指望一个跨产品通用分数解决品味和工程质量。

第四，greenfield 和 brownfield 的迁移成本差异很大。从零项目约束少，长跑 Harness 可以规定栈和风格；存量项目已有代码、测试、设计系统、部署流程和团队惯例，需要项目化 Rubric、现有栈意识、分支与 PR 控制。第五，团队级 Harness 需要所有权、可观测性、version control、commit、PR 和 worktree 这些基础设施承接。第六，human-in-the-loop 的价值在于少量高杠杆插入点：stop condition、权限门、方向修正和失败后 trace 阅读。讲者最后的建议很硬：读完整 trace，理解模型当时的上下文，把学到的失败模式写回 Harness。长程 Agent 的可靠性来自证据边界、上下文隔离、项目约束和人类复盘的持续收紧，一次神奇 prompt 扛不住这种时间尺度。

9 总结：让 Agent 长程可信的三层结构

9.1 三层结构：能力、Harness、证据

整场分享从 00:00:14 的开场走到 01:15:40 的最后一个 trace 问题，核心脉络很清楚：长程 Agent 的可信度来自三层结构共同工作。第一层是模型能力层，负责规划、上下文处理、工具选择、视觉判断和长时间保持连贯性；第二层是 Harness 层，负责把模型放进 planner、generator、evaluator、contract、rubric、handoff、checkpoint 或 compaction 这些外部结构里；第三层是证据层，负责让判断接触真实环境，包括 Playwright MCP、Claude for Chrome MCP、computer use、运行 trace、文件系统状态、截图、合同 criteria 和必要的人类复核。

这三层像一条工程流水线里的材料、夹具和质检记录。模型能力层决定材料本身能承受多复杂的加工；Harness 层决定材料怎么被夹住、分段、测量和交接；证据层决定最后的合格判断有没有碰到真实产品。只有模型能力层，Agent 容易把一个存在于界面上的按钮误认为完整功能。只有 Harness 层，弱模型会在每个角色里继续产生软判断。只有证据层，系统会变成一堆日志审计，缺少能推动下一步生成的执行结构。

可以把本讲的长程 Agent 流程压缩成一个很轻的表达式：

$$P \rightarrow (G \leftrightarrow E \text{ on } C) \rightarrow A \rightarrow T$$

其中， P 表示高层 Planner， G 表示 Generator / Builder， E 表示 Evaluator / Critic， C 表示双方先谈好的 contract criteria， A 表示落到文件系统、浏览器或应用里的 artifact， T 表示运行 trace。这个式子只做压缩呈现，把前文的角色、契约、证据和复盘放到一条链上：先给方向，再让生成器和评估器围绕标准拉扯，接着产生可检查产物，最后通过 trace 反过来修正 Harness。

Ash 在 00:39:00–00:39:53 的收束里给了五条判断，正好对应这三层的边界。自我评估很危险，所以 Harness 层要把 builder 和 evaluator 分开；compaction 可以延长会话，却不能自动保证 coherence，因为有损摘要会漂移；结构化 handoff 和干净上下文能减少角色之间的污染；主观质量可以评分，只要把品味、例子和反例写进 rubric；读 trace 之后再决定哪些 scaffold 该保留、哪些该移走。这里没有“万能脚手架”。脚手架服务于当下模型的短板，模型一进步，某些旧循环就会变成摩擦。

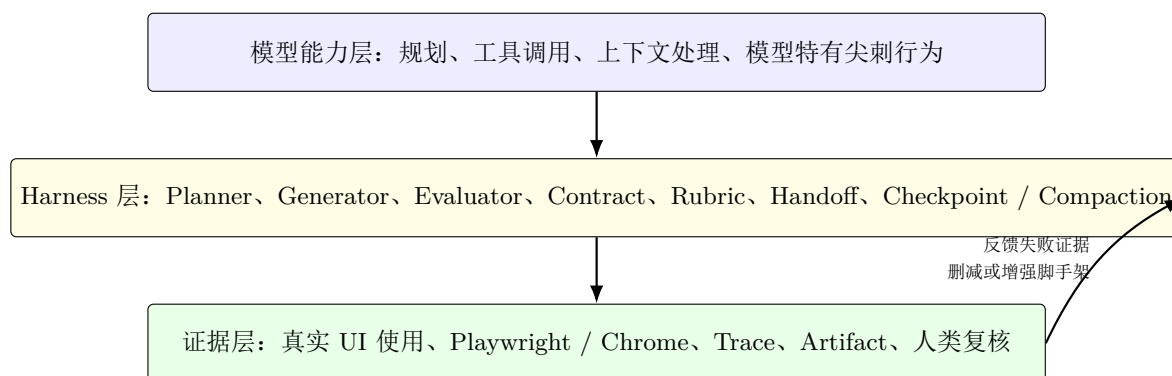


图 25：长程 Agent 可信性的三层结构：模型给出能力上限，Harness 管理角色与状态，证据层逼迫系统面对真实行为。

9.2 从一句 Prompt 到可持续工程流程

这场 talk 一直在反复打碎一个幻想：长程 Agent 的问题不靠一句更聪明的 prompt 解决。Andrew 在 00:04:14–00:05:34 先把杠杆分成模型权重和 Harness；Ash 在 00:17:28 之后把当前模式落到 adversarial evaluator、rubric、contract negotiation 和文件系统 handoff。到 Q&A，听众的问题也从“能不能跑”转成“如何复用、如何信任、如何重跑、如何在 brownfield 里落地”。

复用问题出现在 00:40:18–00:41:20。Ash 的回答很直接：目标是抓住模型弱点里的通用模式，避免只调出某个项目的私房 prompt。前端设计的例子里，团队把“好设计”和“AI slop”都写成例子，rubric 因而能从 web app 泛化到相邻任务。这里的要点是，复用单位应落在“模型常犯什么错、评估器该看什么证据、什么例子代表通过或失败”，一段神秘措辞撑不起复用。

compaction 和 fresh session 的选择在 00:41:28–00:43:50 被问到。Andrew 说，1M context window 已经 GA，模型更 agentic，Opus 4.6 的 generator-evaluator 模式可以使用单个长会话加 compaction；但是否使用多个 fresh sessions 仍取决于用例和 evals。Ash 补了一刀：context rot 是当前模型的失败模式之一，下一代模型可能削弱这个问题，所以他会盯着新模型发布，把已不再必要的结构拿掉。这说明 compaction 是工具，coherence 是目标，两者不能混写。

信任问题在 00:43:53–00:45:39 被问到。Ash 推荐 Playwright MCP 或 Claude for Chrome MCP；他说“想看着浏览器跑”本质上是信任缺口。早期观察有价值，因为人要读 trace、看网络错误、看 console error、看页面有没有文字重叠；长期目标则是让 evaluator 能够独立打开产品、点击、缩放、检查视觉问题和行为问题，然后把证据反馈给 builder。证据层让“我觉得它完成了”变成“它真的跑过这条路径”。

重跑和丢弃问题在 00:45:45–00:48:37 很关键。听众问，如果用户拿到一个游戏后说“make it better”，generator-evaluator 会继续改，还是 evaluator 会草草放行。Ash 观察到，在 4.6 系列模型和这套模式下，系统非常愿意丢掉已经做了十轮的尝试，从头开始；如果 evaluator 发现当前路线无法沿 rubric 有效爬坡，它会要求 builder 抛弃旧做法。这个细节说明 adversarial evaluator 的价值不只在挑错，也在阻止 builder 对自己刚写的结构产生保护欲。

长程系统里最危险的状态是“失败后继续补同一个坏结构”；明显失败反而容易止损。如果 evaluator 没有证据、没有 rubric、没有权限要求重启，系统会把局部修补伪装成进展。实践中应允许 evaluator 明确说：这条路线无法满足 contract，请重新开局。

planner 的边界在 00:49:06–00:52:01 被追问。Ash 说，他们会把 PM / planner 生成的主 spec 定期放回 builder 和 evaluator 的上下文，作为“到底要建什么”的参照；但 planner 不应成为核心循环。planner 画外轮廓，builder 和 evaluator 负责把轮廓压成 feature set、tests 和 contracts。若让 planner

频繁介入，会把高层方向和局部修复搅在一起，反而削弱 contract negotiation 的清晰度。

9.3 实践清单：从最小可用 Harness 开始

面向读者的落地顺序可以很朴素。第一，先选一个领域。不要一开始就做横跨浏览器、后端、设计系统、支付和运维的巨型任务；选 web app、内部工具、数据清洗、测试生成或文档维护中的一个，才能看清模型弱点。第二，写一份 rubric。rubric 要包含维度、通过样例、失败样例和边界样例；设计品味、API 设计、代码质量、可访问性、错误处理都可以分别评分。

第三，把 builder 和 evaluator 拆开。它们要有不同的上下文、不同的系统提示、不同的任务动机。Evaluator 应该看产物、看 handoff、看 contract 和证据，不要吃进 builder 的全部思路。Ash 在 01:00:05–01:01:06 说过，他们试过让 critic 拿到 generator 的上下文，但这种做法会把两个模型流的错误想法搅浑；更有效的方式是让 evaluator 直接判断输出，指出“这里有问题”，再让 generator 自己定位修复路径。

第四，要求 contract negotiation 发生在编码前。Builder 先说会建什么、如何验证；evaluator 追问测试是否足够、边界是否覆盖、用户故事是否可证。第五，记录 trace 和 artifact。文件系统很适合做共享状态：Ash 在 00:54:36–00:56:06 提到，可让 harness 写 JSON 面包屑，记录“尝试了什么、评估发现什么 bug、做了什么 fix、fix 是否通过”，再保留高层文件结构说明，方便人类或 Claude Code 接着工作。

第六，专门复盘失败。不要只保存成功样本。Ash 在 01:10:03–01:10:25 描述了他们会启动多次生成，比如十次里三次成功、七次失败，然后坐下来读失败 trace，调整主 harness prompt，再试，直到愿意让系统自主运行。第七，随着模型改进主动简化 Harness。00:34:53–00:36:08 的例子很明确：Opus 4.5 需要更多 context reset 和 sprint decomposition；Opus 4.6 能更连贯地支撑约 2 小时连续构建，于是团队减少强制 fresh session，调整 evaluator 的运行频率，保留 planner-generator-evaluator 核心和文件系统 handoff。

最小可用 Harness 的检查表可以写成七句话：选一个领域；写 rubric；拆 builder 和 evaluator；编码前谈 contract；让 evaluator 用真实工具拿证据；保存 trace、artifact 和 JSON 状态；定期读失败并删掉过时脚手架。每一句都对应 talk 里的一个具体故障来源：自评偏差、上下文漂移、证据不足、主观品味无标准、旧 Harness 与新模型能力不匹配。

9.4 边界与开放问题

限制也要说硬。第一，greenfield 适配更强，brownfield 要更谨慎。01:11:00–01:12:06 的回答里，Andrew 说这套特定模式更适合 brand new applications；brownfield 需要更多控制、项目化 rubric、已有测试、既有技术栈理解和代码库约束。React、Postgres、Node 的默认组合不能直接套到完全不同的存量系统里。对 brownfield，更现实的路径可能是自动监控生成 issue、Agent 开 PR、已有 review 流程把关，再逐步扩展到更长链路。

第二，compaction 会丢信息。00:39:06–00:39:15 的 closing 明说 lossy summaries drift。压缩能让会话继续，却会把细节、置信度、失败证据和未完成事项压平。结构化 handoff、外部 artifact 和 trace 是补偿手段；它们也会出错，所以关键状态要写成可检查的文件、criteria 或测试结果。

第三，rubric 需要人类判断和例子。01:02:51–01:05:29 的“能不能更科学”问题里，Ash 说他们会把 rubric 写得很细：设计品味、API 设计、代码质量等可以各自形成 criteria，evaluator 负责推动 builder 沿这些维度爬坡。但他也承认，这种分数很难跨产品、跨 run 直接比较。rubric 的价值首先在一个产品或一次运行内部，帮助系统知道“从哪里开始、往哪里改、什么时候该停”。

第四, evaluator 必须使用证据。只让 evaluator 读 diff, 仍然可能复制 builder 的错觉。前端任务要打开页面、点击路径、看 console 和 network; native app 要用 computer use; 长程任务要读 trace; 多 Agent 任务要检查 handoff 文件。01:13:25–01:15:13 的最后一个问题把这一点落到方法论: Ash 回答 “read the whole thing”, 并说读 trace 要把自己放进模型的位置, 理解它当时看到什么、没看到什么、为什么会那样判断。Andrew 补充说, 要把学到的东西写回 prompt templates、Claude.md 或 skills。

第五, human-in-the-loop 不能当偷懒按钮。01:07:42–01:10:43 的 scrum review 问答里, Ash 承认今天并没有把这套技术用于所有事情; 若需要人类介入, 可以用 hooks 注入 stop condition, 让 evaluator 把控制权交回人类, 接收开发者消息后继续。但他们的目标仍然是把失败模式吸收到 Harness 内部, 避免每次不稳就固定插一个人工检查点。人类最该出现的位置, 是 rubric 设计、失败样本复盘、边界确认和最终合并决策。

这套结构没有免检通行证。它提高的是可观察性、可纠错性和可复盘性。若 evaluator 不看真实产物, rubric 没有人类例子, trace 没人读, brownfield 约束没有进入上下文, 长程 Agent 仍会用漂亮的过程文件掩盖坏结果。

9.5 本章小结

本章把全场内容压成三层: 模型能力层提供规划、工具使用、上下文处理和连贯性的基础; Harness 层用 planner、generator、evaluator、contract、rubric、handoff、checkpoint 与 compaction 把任务外部化; 证据层用真实 UI、工具结果、trace、artifact、criteria 和人类复核校正判断。三层缺一层, 长程 Agent 都会在数小时尺度上漂移。

Ash 的 closing 和 Q&A 给出的最终建议很务实: 自评是陷阱, 拆出 adversarial evaluator; compaction 可以用, 但不要把它当成 coherence; 结构化 handoff 和干净上下文能降低污染; 主观质量写进 rubric 后可以被推动; 读完整 trace 后再调整 prompt、skills、Claude.md 和 Harness; 模型进步后, 主动删掉已经失去必要性的 scaffold。对读者来说, 起步方式很简单: 从一个领域、一份 rubric、一对 builder/evaluator、一次 contract negotiation、一套 trace 记录开始, 然后用失败样本把这套系统磨到能被信任。